

Putting Theory Into Practice

Apache DataFusion & Apache DataFusion Comet

Matthew Butrovich | February 18, 2025

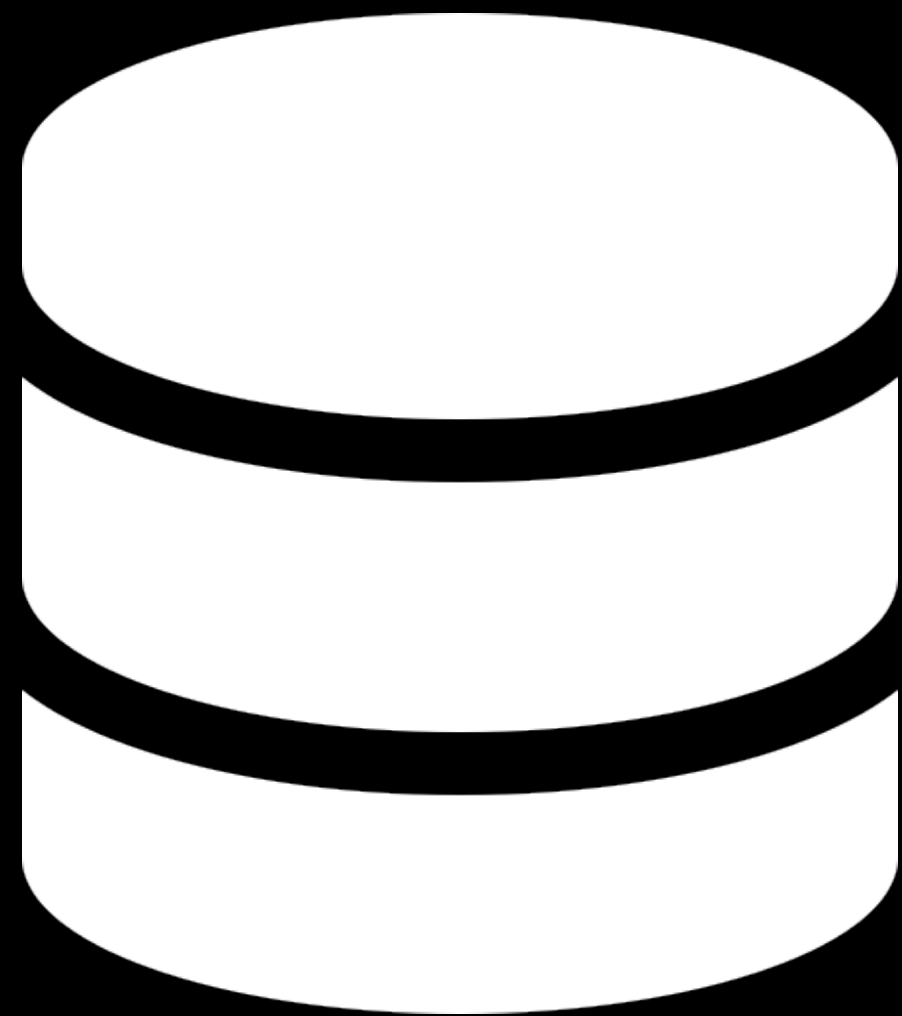
This talk was prepared by Matt Butrovich in his personal capacity. The opinions expressed in this talk are the author's own and do not reflect the views of The Apache Software Foundation, Carnegie Mellon University, or Apple Inc.

About Me

<https://mattbutrovi.ch>

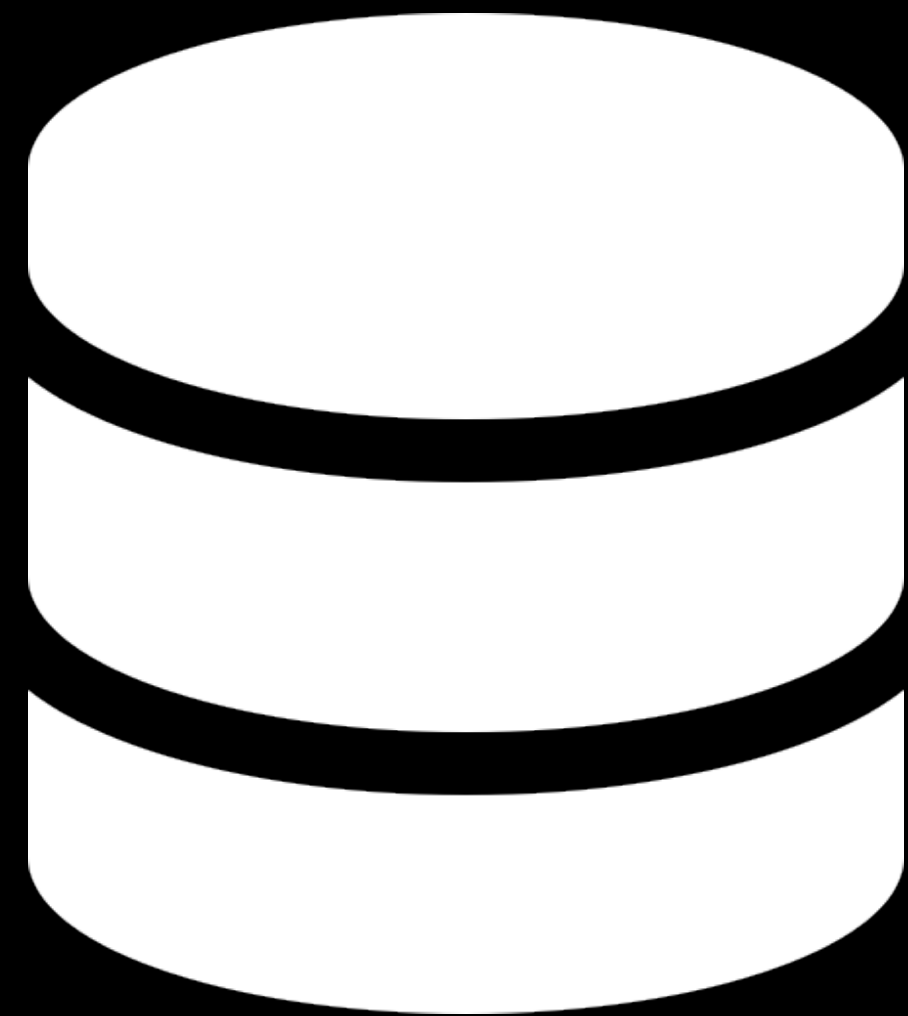
About Me

<https://mattbutrovi.ch>



About Me

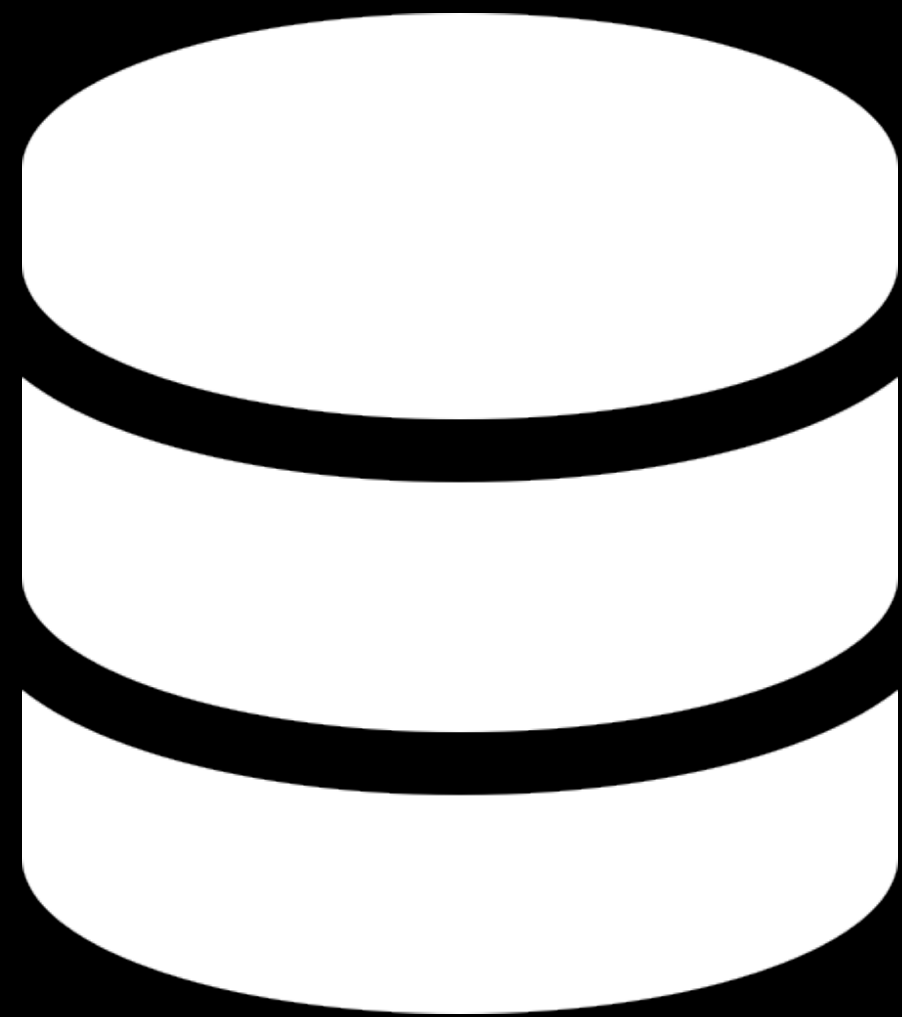
<https://mattbutrovi.ch>



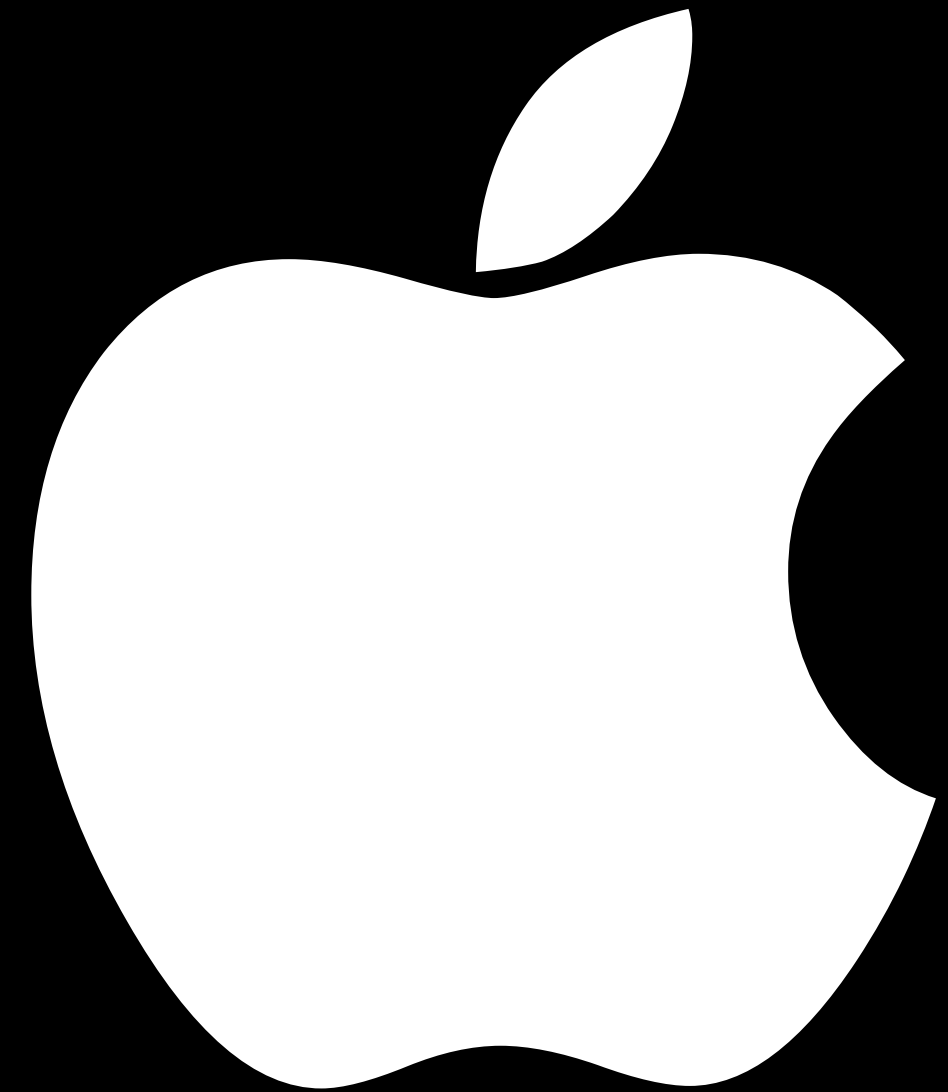
Carnegie
Mellon
University

About Me

<https://mattbutrovi.ch>



Carnegie
Mellon
University



Apache DataFusion

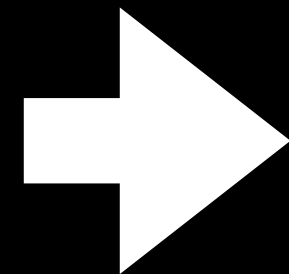
Embedded Analytic Query Engine

Apache DataFusion

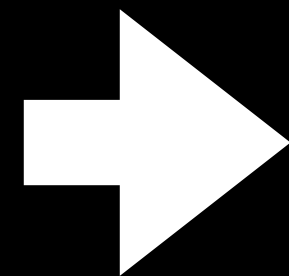
Embedded Analytic Query Engine

Apache DataFusion

User Queries
(SQL, DataFrames, ...)

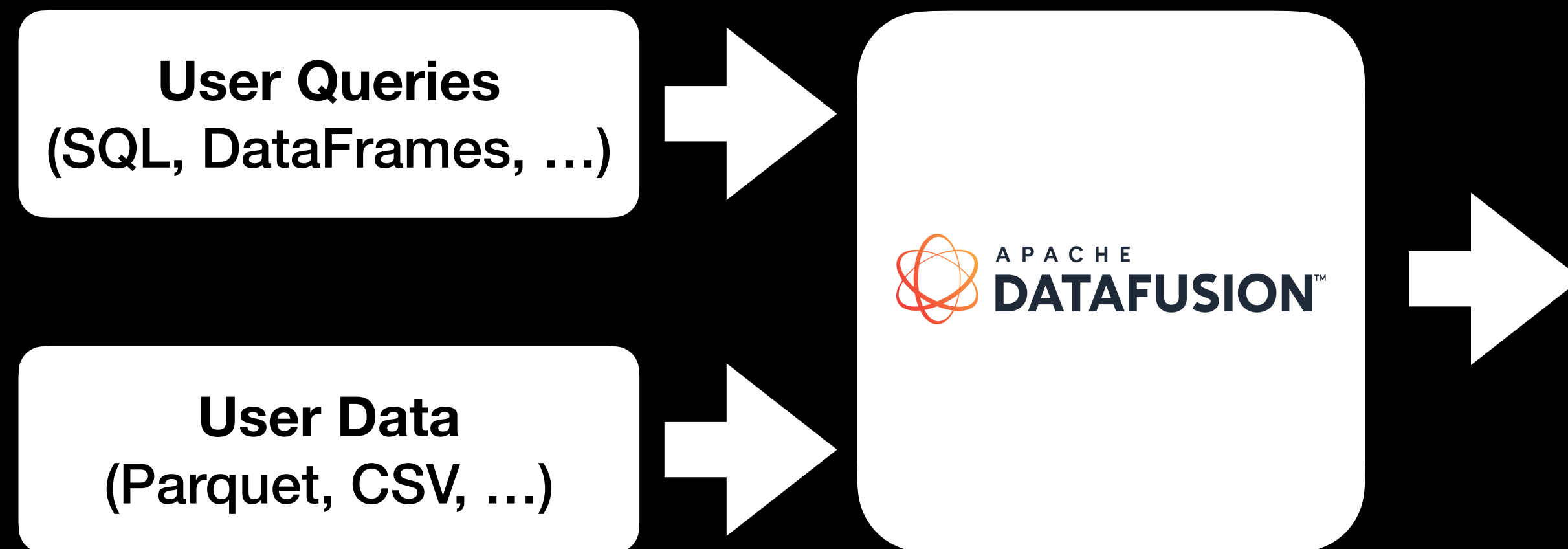


User Data
(Parquet, CSV, ...)



Embedded Analytic Query Engine

Apache DataFusion



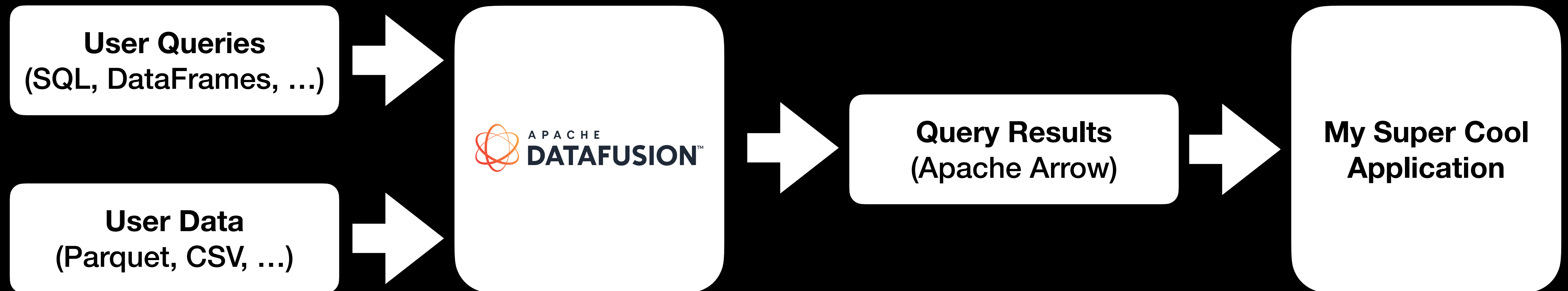
Embedded Analytic Query Engine

Apache DataFusion



Embedded Analytic Query Engine

Apache DataFusion



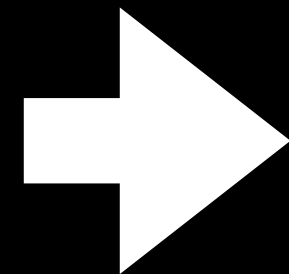
Embedded Analytic Query Engine

Apache DataFusion

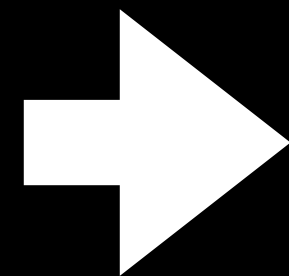
Embedded Analytic Query Engine

Apache DataFusion

User Queries
(SQL, DataFrames, ...)

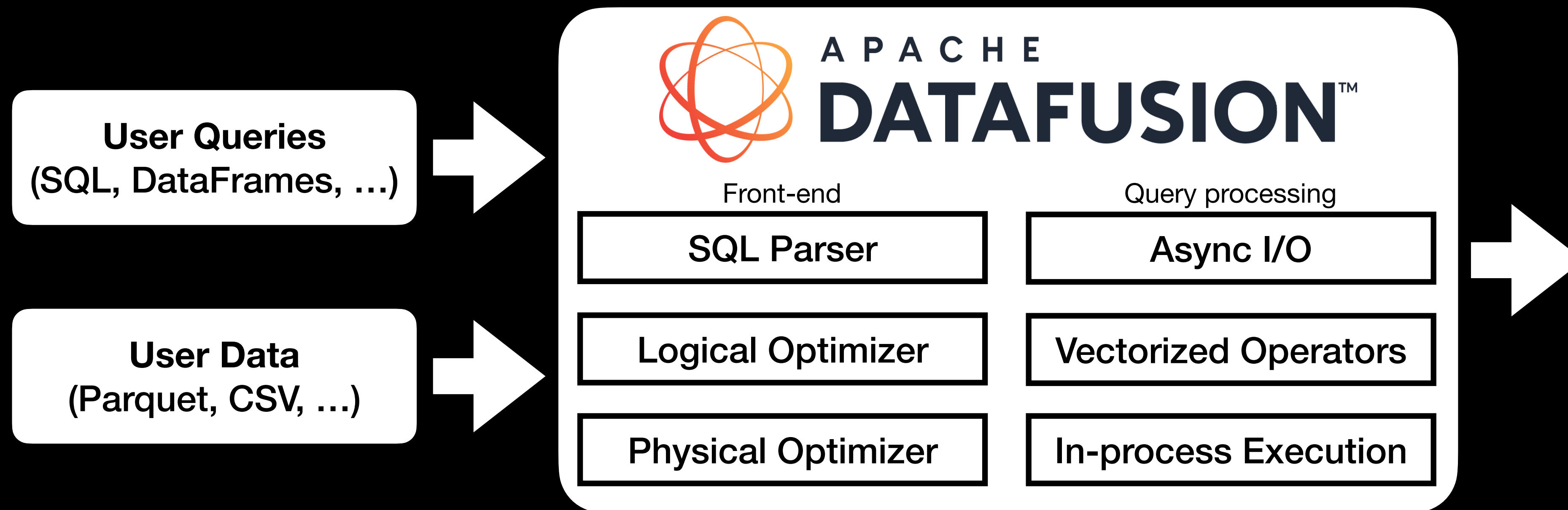


User Data
(Parquet, CSV, ...)



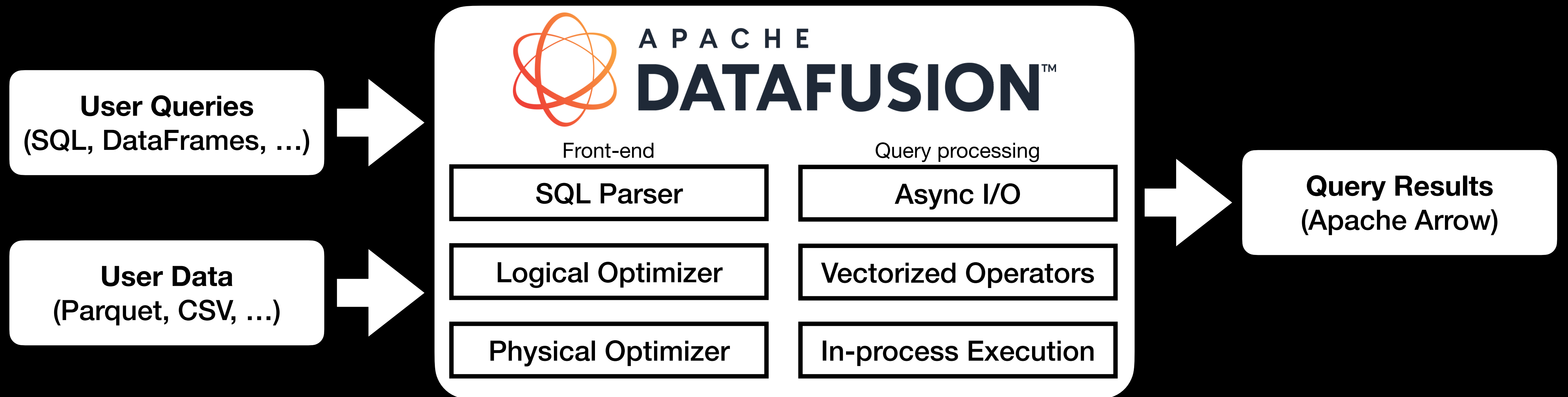
Embedded Analytic Query Engine

Apache DataFusion



Embedded Analytic Query Engine

Apache DataFusion



History

Apache DataFusion

History

Apache DataFusion

- Began as open-source
 - Created by Andy Grove

History

Apache DataFusion

- Began as open-source
 - Created by Andy Grove
- Donated as an Apache Arrow subproject, now a top-level ASF project

History

Apache DataFusion

- Began as open-source
 - Created by Andy Grove
- Donated as an Apache Arrow subproject, now a top-level ASF project
- GitHub repository:
 - 729 contributors
 - Almost 10,000 commits

History

Apache DataFusion

- Began as open-source
 - Created by Andy Grove
- Donated as an Apache Arrow subproject, now a top-level ASF project
- GitHub repository:
 - 729 contributors
 - Almost 10,000 commits

Apache Arrow DataFusion: a Fast, Embeddable, Modular Analytic Query Engine

Andrew Lamb
InfluxData
Boston, MA, USA
alamb@apache.org

Yijie Shen
Space and Time
Irvine, CA, USA
yjshen@apache.org

Daniël Heres
Coralogix
Utrecht, The Netherlands
dheres@apache.org

Jayjeet Chakraborty
UC Santa Cruz
Santa Cruz, CA, USA
jayjeetc@ucsc.edu

Mehmet Ozan Kabak
Synnada
Austin, TX, USA
ozankabak@apache.org

Chao Sun
Apple
Cupertino, CA, USA
sunchao@apache.org

Liang-Chi Hsieh
Apple
Seattle, WA, USA
viirya@apache.org

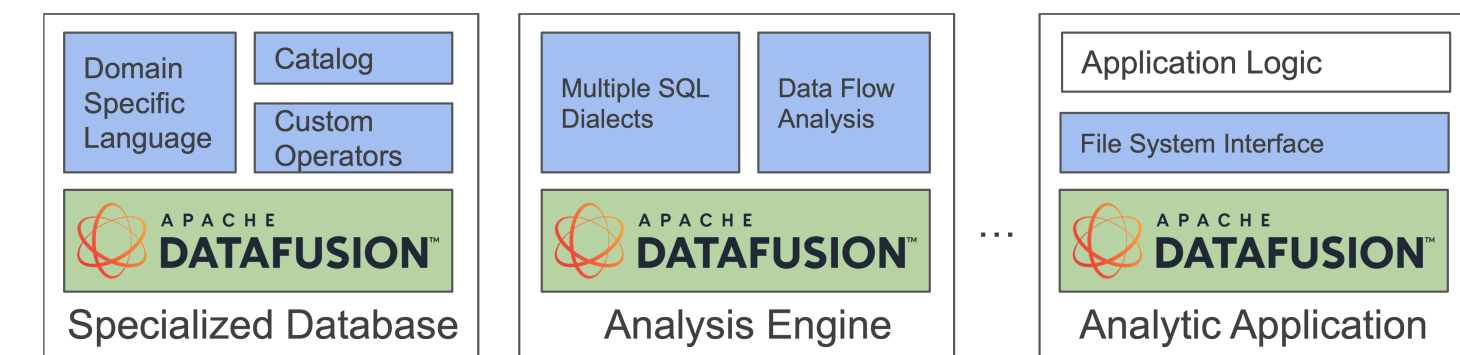


Figure 1: When building with DataFusion, system designers implement domain-specific features via extension APIs (blue), rather than re-implementing standard OLAP query engine technology (green).

ABSTRACT

Apache Arrow DataFusion[25] is a fast, embeddable, and extensible query engine written in Rust[76] that uses Apache Arrow[24] as its memory model. In this paper we describe the technologies on which it is built, and how it fits in long-term database implementation trends. We then enumerate its features, optimizations, architecture and extension APIs to illustrate the breadth of requirements of modern OLAP engines as well as the interfaces needed by systems built with them. Finally, we demonstrate open standards and extensible design do not preclude state-of-the-art performance using a series of experimental comparisons to DuckDB[66].

While the individual techniques used in DataFusion have been previously described many times, it differs from other industrial strength engines by providing competitive performance *and* an open architecture that can be customized using more than 10 major extension APIs. This flexibility has led to use in many commercial and open source databases, machine learning pipelines, and other

data-intensive systems. We anticipate that the accessibility and versatility of DataFusion, along with its competitive performance, will further the proliferation of high-performance custom data infrastructures tailored to specific needs assembled from modular components[18, 61].

CCS CONCEPTS

• **Information systems** → **Database management system engines**; **Online analytical processing engines**; **DBMS engine architectures**; Relational database model; *Database query processing*; • **Software and its engineering** → *Abstraction, modeling and modularity*; *Software performance*; *Software usability*.

KEYWORDS

Database Systems, Modular Query Engines, Column Stores, OLAP, Vectorized Execution, Parallel Execution, API Design

ACM Reference Format:

Andrew Lamb, Yijie Shen, Daniël Heres, Jayjeet Chakraborty, Mehmet Ozan Kabak, Chao Sun, and Liang-Chi Hsieh. 2024. Apache Arrow DataFusion: a Fast, Embeddable, Modular Analytic Query Engine. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion '24)*, June 9–15, 2024, Santiago, AA, Chile. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3626246.3653368>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD-Companion '24, June 9–15, 2024, Santiago, AA, Chile
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0422-2/24/06.
<https://doi.org/10.1145/3626246.3653368>

Applications

Apache DataFusion

Applications

Apache DataFusion

- Query engine for building new DBMSs
 - InfluxDB, ParadeDB, GreptimeDB, ...

Applications

Apache DataFusion

- Query engine for building new DBMSs
 - InfluxDB, ParadeDB, GreptimeDB, ...
- Query accelerator for existing DBMSs
 - DataFusion Comet, Blaze, ...

Applications

Apache DataFusion

- Query engine for building new DBMSs
 - InfluxDB, ParadeDB, GreptimeDB, ...
- Query accelerator for existing DBMSs
 - DataFusion Comet, Blaze, ...
- Scriptable CLI for batch data lake jobs
 - Data formats: Parquet, Vortex, LanceDB, Iceberg, Delta

Applications

Apache DataFusion

- Query engine for building new DBMSs
 - InfluxDB, ParadeDB, GreptimeDB, ...
- Query accelerator for existing DBMSs
 - DataFusion Comet, Blaze, ...
- Scriptable CLI for batch data lake jobs
 - Data formats: Parquet, Vortex, LanceDB, Iceberg, Delta
- Python bindings for Jupyter notebooks

Performance

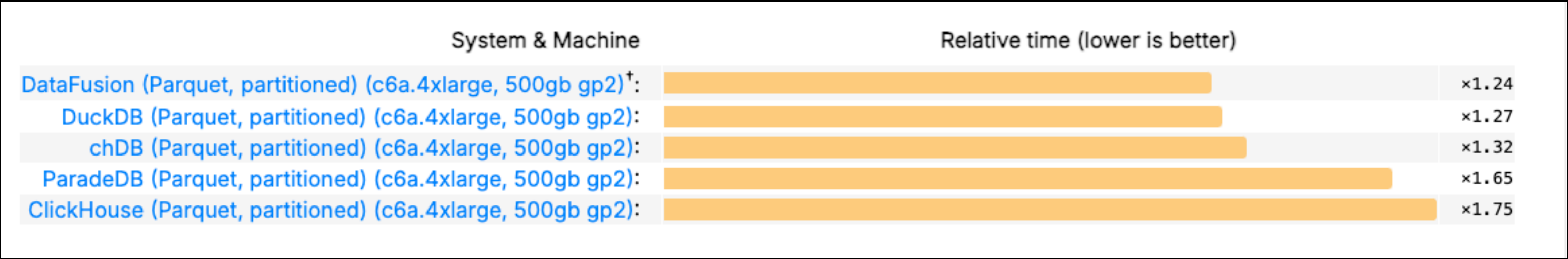
Apache DataFusion

Source: [Apache DataFusion is now the fastest single node engine for querying Apache Parquet files](#)

Performance

Apache DataFusion

ClickBench, single-node, partitioned Parquet files:

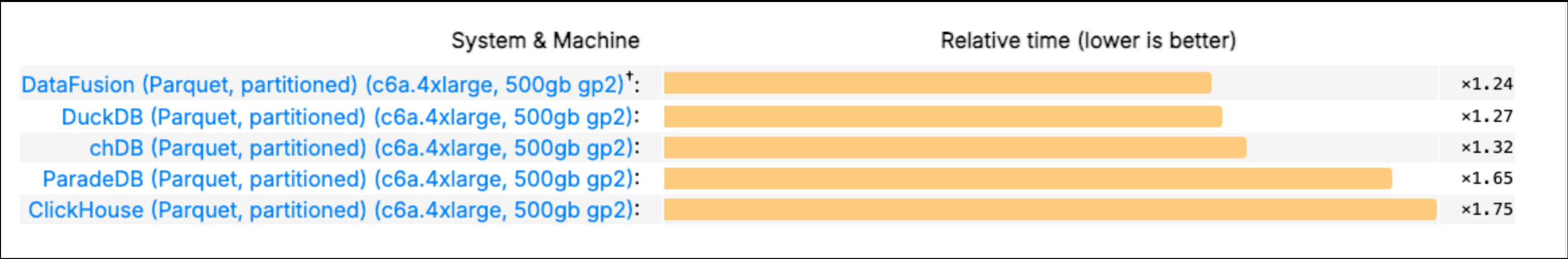


Source: [Apache DataFusion is now the fastest single node engine for querying Apache Parquet files](#)

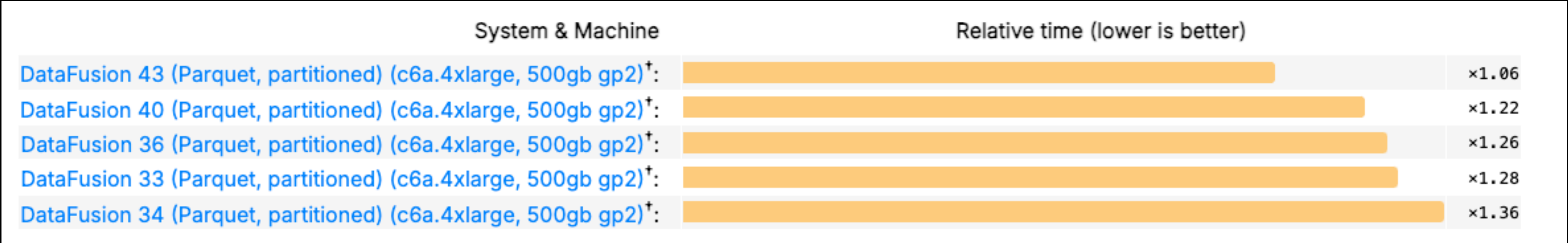
Performance

Apache DataFusion

ClickBench, single-node, partitioned Parquet files:



Performance over time:



Source: [Apache DataFusion is now the fastest single node engine for querying Apache Parquet files](#)

German-Style Strings

Apache DataFusion

German-Style Strings

Apache DataFusion

- Data representations impact performance!

German-Style Strings

Apache DataFusion

- Data representations impact performance!
- Implemented in DataFusion as `StringView`

German-Style Strings

Apache DataFusion

- Data representations impact performance!
- Implemented in DataFusion as `StringView`
- Benefits:
 - Faster loading from Parquet files
 - Faster filtering of values

German-Style Strings

Apache DataFusion

German-Style Strings

Apache DataFusion

- TU Munich DBMSs:
 - HyPer, acquired by Tableau, later acquired by Salesforce
 - Umbra, commercialized as CedarDB

German-Style Strings

Apache DataFusion

- TU Munich DBMSs:
- HyPer, acquired by Tableau, later acquired by Salesforce
- Umbra, commercialized as CedarDB

Efficiently Compiling Efficient Query Plans for Modern Hardware

Fast Serializable Multi-Version Concurrency Control for Main-Memory Database Systems

Thomas Neumann, Michael Freitag
Technische Universität München
{neumann,freitag}@in.tum.de

ABSTRACT

The increases in main-memory sizes over the last decade have made pure in-memory database systems feasible, and in-memory systems offer unprecedented performance. However, DRAM is still relatively expensive, and the growth of main-memory sizes has slowed down. In contrast, the prices for SSDs have fallen substantially in the last years, and their read bandwidth has increased to gigabytes per second. This makes it attractive to combine a large in-memory buffer with fast SSDs as storage devices, combining the excellent performance for the in-memory working set with the scalability of a disk-based system.

In this paper we present the Umbra system, an evolution of the pure in-memory HyPer system towards a disk-based, or rather SSD-based, system. We show that by introducing a novel low-overhead buffer manager with variable-size pages we can achieve comparable performance to an in-memory database system for the cached working set, while handling accesses to uncached data gracefully. We discuss the changes and techniques that were necessary to handle the out-of-memory case gracefully and with low overhead, offering insights into the design of a memory optimized disk-based system.

1. INTRODUCTION

Hardware trends have greatly affected the development and evolution of database management systems over time. Historically, most of the data was stored on (rotating) disks, and only small fractions of the data could be kept in RAM in a buffer pool. As main memory sizes grew significantly, up to terabytes of RAM, this perspective changed as large fractions of the data or even all data could now be kept in memory. In comparison to disk-based systems, this offered a huge performance advantage and led to the development of pure in-memory database systems [4, 5], including our own system HyPer [9]. These systems make use of RAM-only storage and offer outstanding performance, but tend to fail or degrade heavily if the data does not fit into memory.

Moreover, we currently observe two hardware trends that cast strong doubt on the viability of pure in-memory systems. First, RAM sizes are not increasing significantly any more. Ten years

ago, one could conceivably buy a commodity server with 1 TB of memory for a reasonable price. Today, affordable main memory sizes might have increased to 2 TB, but going beyond that disproportionately increases the costs. As costs usually have to be kept under control though, this has caused the growth of main memory sizes in servers to subside in the recent years.

On the other hand, SSDs have achieved astonishing improvements over the past years. A modern 2 TB M.2 SSD can read with about 3.5 GB/s, while costing only \$500. In comparison, 2 TB of server DRAM costs about \$20,000, i.e. a factor of 40 more. By placing multiple SSDs into one machine we can get excellent read bandwidths at a fraction of the cost of a pure DRAM solution. Because of this, Lomet argues that pure in-memory DBMSs are uneconomical [15]. They offer the best possible performance, of course, but they do not scale beyond a certain size and are far too expensive for most use cases. Combining large main memory buffers with fast SSDs, in contrast, is an attractive alternative as the cost is much lower and performance can be nearly as good.

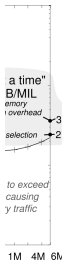
We wholeheartedly agree with this notion, and present our novel Umbra system which simultaneously features the best of both worlds: Genuine in-memory performance on the cached working set, and transparent scaling beyond main memory where required. Umbra is the spiritual successor of our pure in-memory system HyPer, and completely eliminates the restrictions of HyPer on data sizes. As we will show in this paper, we achieve this without sacrificing any performance in the process. Umbra is a fully functional general-purpose DBMS that is actively developed further by our group. All techniques presented in this paper have been implemented and evaluated within this working system. While Umbra and HyPer share several design choices like a compiling query execution engine, Umbra deviates in many important aspects due to the necessities of external memory usage. In the following, we present key components of the system and highlight the changes that were necessary to support arbitrary data sizes without losing performance for the common case that the entire working set fits into main memory.

A key ingredient for achieving this is a novel buffer manager that combines low-overhead buffering with *variable-size pages*. Compared to a traditional disk-based system, in-memory systems have the major advantage that they can do away with buffering, which both eliminates overhead and greatly simplifies the code. For disk-based systems, common wisdom dictates to use a buffer manager with fixed-size pages. However, while this simplifies the buffer manager itself, it makes using the buffer manager exceedingly difficult. For example, large strings or lookup tables for dictionary compression often cannot easily be stored in a single fixed-size page, and both complex and expensive mechanisms are thus required all over the database system in order to handle large objects. We ar-

per

It is hard to imagine serializability king (2PL) [42]. It would write locks to be executed in a well-execution scheduler disadvantages: r. Second, most re harmless from ; a locking-based on is allowed to by a potentially to wait until the its the degree of

VCC) [42, 3, 28]. Instead of up-creates a new ver-rent readers can transaction pro-ad-only transac-not have to use ble property and (VCC, e.g., Orana [10, 37], and at use MVCC do er isolation level transaction sees e last committed and the DBMS o not update the y good isolation, owed [1, 2]. This ng SI serializable n particular, the re entire read set verhead for read-it is desirable to ad to silent data o-detect bugs. ay to implement for SI and for full admittedly more VCC is a well un-newed interest in Careful engineer-of version main-ry processing. It



cution engines)

oor code locality n by considering on. As the tuples scan operator has eam the current ompression code

arn systems to a either internally ber of tuples at pressed data), or during each next e [1]. This block-f calling another ced tuples, such However, it also odel, namely the that an operator hout copying or ns, for example, ss tuples around mplex operators ie of their input ple during a call any more, as the to be accessible. like allowing for lack of pipelin mory bandwidth. it is that a hand-ry fast vectorized from [16]). In a human might use would never come

This article is published under a Creative Commons Attribution License (<http://creativecommons.org/licenses/by/3.0/>), which permits distribution and reproduction in any medium as well as allowing derivative works, provided that you attribute the original work to the author(s) and CIDR 2020. 10th Annual Conference on Innovative Data Systems Research (CIDR '20). January 12-15, 2020, Amsterdam, Netherlands.

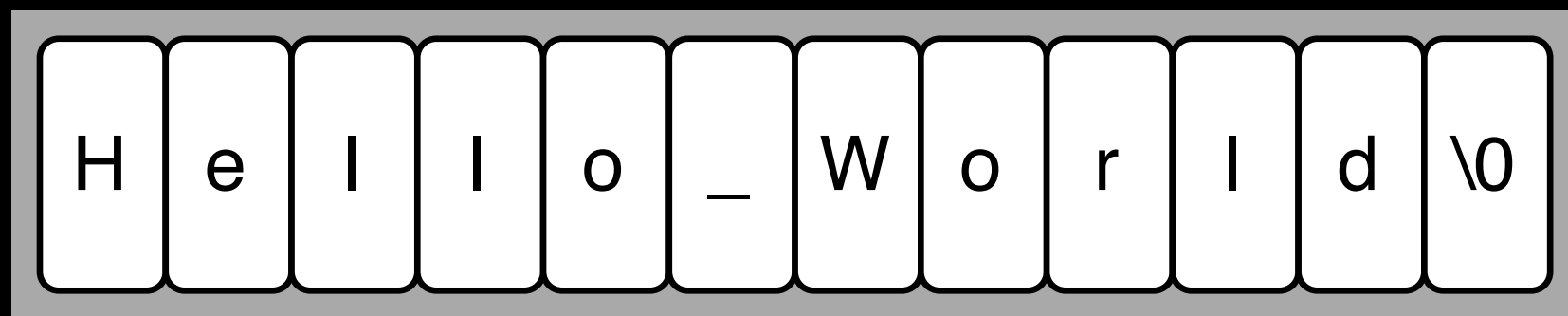
German-Style Strings

Apache DataFusion

German-Style Strings

Apache DataFusion

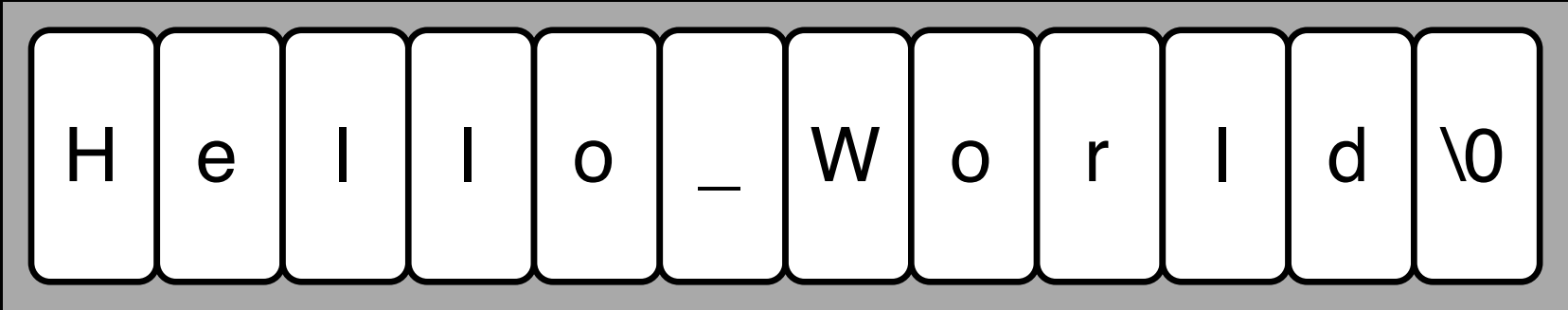
C



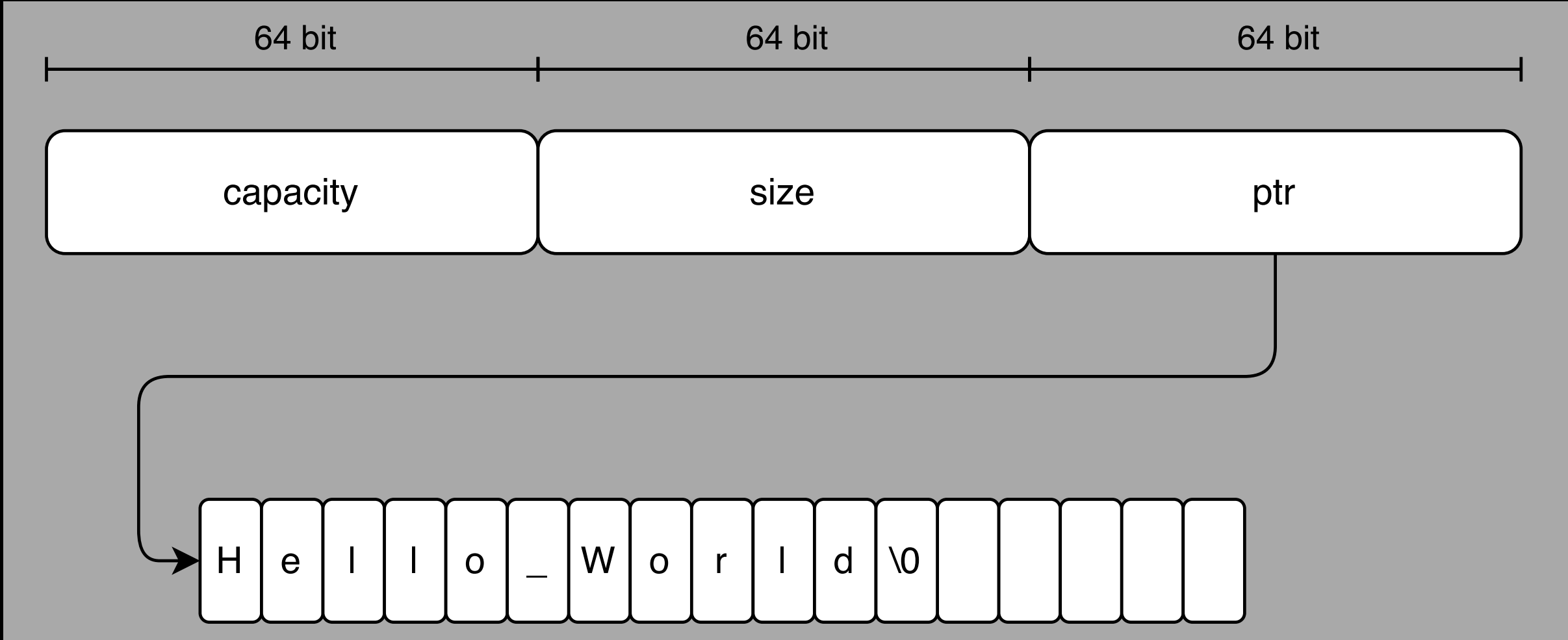
German-Style Strings

Apache DataFusion

C



C++



German-Style Strings

Apache DataFusion

German-Style Strings

Apache DataFusion

- Observations:

German-Style Strings

Apache DataFusion

- Observations:
 - Most strings are short

German-Style Strings

Apache DataFusion

- Observations:
 - Most strings are short
 - Strings are not changed often

German-Style Strings

Apache DataFusion

- Observations:
 - Most strings are short
 - Strings are not changed often
 - We usually only look at a subset of the string

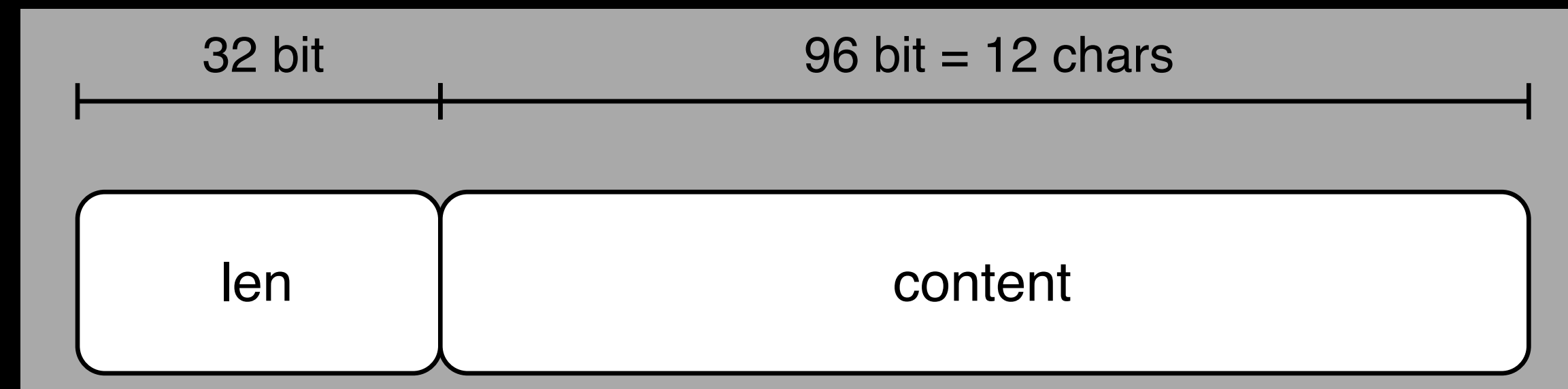
German-Style Strings

Apache DataFusion

German-Style Strings

Apache DataFusion

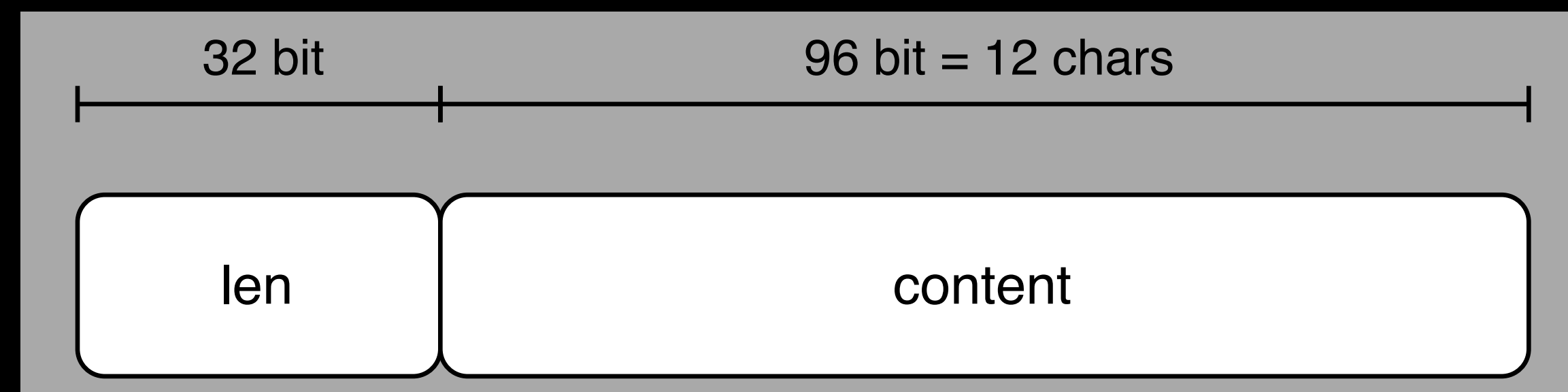
Short string



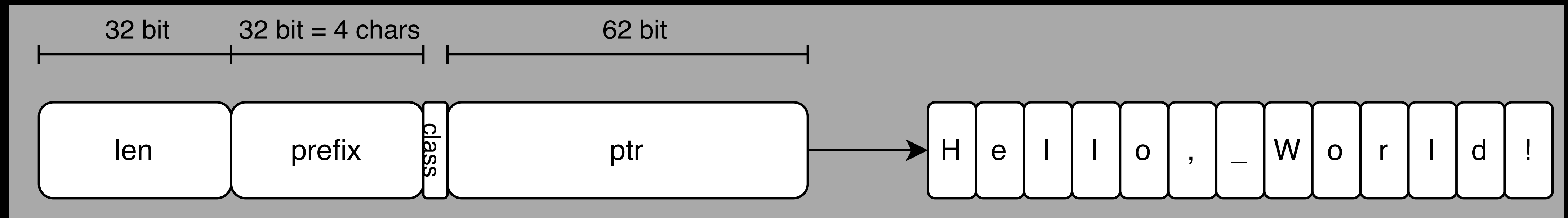
German-Style Strings

Apache DataFusion

Short string



Long string

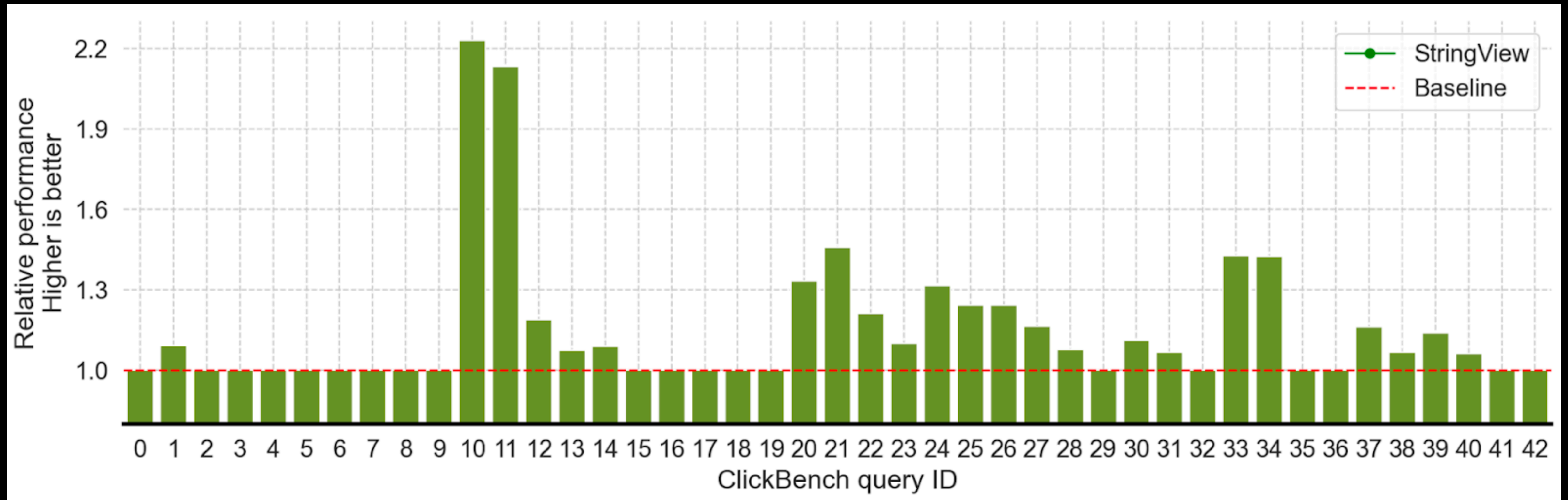


German-Style String Performance

Apache DataFusion

German-Style String Performance

Apache DataFusion



Source: [Faster DataFusion with StringView](#)

Apache DataFusion Comet

Apache Spark

Apache DataFusion Comet



Spark SQL: Relational Data Processing in Spark

Michael Armbrust[†], Reynold S. Xin[†], Cheng Lian[†], Yin Huai[†], Davies Liu[†], Joseph K. Bradley[†],
Xiangrui Meng[†], Tomer Kaftan[‡], Michael J. Franklin^{†‡}, Ali Ghodsi[†], Matei Zaharia^{†*}

[†]Databricks Inc. ^{*}MIT CSAIL [‡]AMPLab, UC Berkeley

ABSTRACT

Spark SQL is a new module in Apache Spark that integrates relational processing with Spark’s functional programming API. Built on our experience with Shark, Spark SQL lets Spark programmers leverage the benefits of relational processing (*e.g.*, declarative queries and optimized storage), and lets SQL users call complex analytics libraries in Spark (*e.g.*, machine learning). Compared to previous systems, Spark SQL makes two main additions. First, it offers much tighter integration between relational and procedural processing, through a declarative `DataFrame` API that integrates with procedural Spark code. Second, it includes a highly extensible optimizer, Catalyst, built using features of the Scala programming language, that makes it easy to add composable rules, control code generation, and define extension points. Using Catalyst, we have built a variety of features (*e.g.*, schema inference for JSON, machine learning types, and query federation to external databases) tailored for the complex needs of modern data analysis. We see Spark SQL as an evolution of both SQL-on-Spark and of Spark itself, offering richer APIs and optimizations while keeping the benefits of the Spark programming model.

Categories and Subject Descriptors

H.2 [Database Management]: Systems

Keywords

Databases; Data Warehouse; Machine Learning; Spark; Hadoop

1 Introduction

Big data applications require a mix of processing techniques, data sources and storage formats. The earliest systems designed for these workloads, such as MapReduce, gave users a powerful, but low-level, procedural programming interface. Programming such systems was onerous and required manual optimization by the user to achieve high performance. As a result, multiple new systems sought to provide a more productive user experience by offering relational interfaces to big data. Systems like Pig, Hive, Dremel and Shark [29, 36, 25, 38] all take advantage of declarative queries to provide richer automatic optimizations.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD’15, May 31–June 4, 2015, Melbourne, Victoria, Australia.
Copyright is held by the owner/author(s). Publication rights licensed to ACM.
ACM 978-1-4503-2758-9/15/05 ...\$15.00.
<http://dx.doi.org/10.1145/2723372.2742797>.

While the popularity of relational systems shows that users often prefer writing declarative queries, the relational approach is insufficient for many big data applications. First, users want to perform ETL to and from various data sources that might be semi- or unstructured, requiring custom code. Second, users want to perform advanced analytics, such as machine learning and graph processing, that are challenging to express in relational systems. In practice, we have observed that most data pipelines would ideally be expressed with a combination of both relational queries and complex procedural algorithms. Unfortunately, these two classes of systems—relational and procedural—have until now remained largely disjoint, forcing users to choose one paradigm or the other.

This paper describes our effort to combine both models in Spark SQL, a major new component in Apache Spark [39]. Spark SQL builds on our earlier SQL-on-Spark effort, called Shark. Rather than forcing users to pick between a relational or a procedural API, however, Spark SQL lets users seamlessly intermix the two.

Spark SQL bridges the gap between the two models through two contributions. First, Spark SQL provides a *DataFrame API* that can perform relational operations on both external data sources and Spark’s built-in distributed collections. This API is similar to the widely used data frame concept in R [32], but evaluates operations lazily so that it can perform relational optimizations. Second, to support the wide range of data sources and algorithms in big data, Spark SQL introduces a novel extensible optimizer called *Catalyst*. Catalyst makes it easy to add data sources, optimization rules, and data types for domains such as machine learning.

The `DataFrame` API offers rich relational/procedural integration within Spark programs. `DataFrames` are collections of structured records that can be manipulated using Spark’s procedural API, or using new relational APIs that allow richer optimizations. They can be created directly from Spark’s built-in distributed collections of Java/Python objects, enabling relational processing in existing Spark programs. Other Spark components, such as the machine learning library, take and produce `DataFrames` as well. `DataFrames` are more convenient and more efficient than Spark’s procedural API in many common situations. For example, they make it easy to compute multiple aggregates in one pass using a SQL statement, something that is difficult to express in traditional functional APIs. They also automatically store data in a columnar format that is significantly more compact than Java/Python objects. Finally, unlike existing data frame APIs in R and Python, `DataFrame` operations in Spark SQL go through a relational optimizer, Catalyst.

To support a wide variety of data sources and analytics workloads in Spark SQL, we designed an extensible query optimizer called Catalyst. Catalyst uses features of the Scala programming language, such as pattern-matching, to express composable rules in a Turing-complete language. It offers a general framework for transforming

Apache Spark

Apache DataFusion Comet

- Large scale distributed query processing



Spark SQL: Relational Data Processing in Spark

Michael Armbrust[†], Reynold S. Xin[†], Cheng Lian[†], Yin Huai[†], Davies Liu[†], Joseph K. Bradley[†], Xiangrui Meng[†], Tomer Kaftan[‡], Michael J. Franklin^{†‡}, Ali Ghodsi[†], Matei Zaharia^{†*}

[†]Databricks Inc. ^{*}MIT CSAIL [‡]AMPLab, UC Berkeley

ABSTRACT

Spark SQL is a new module in Apache Spark that integrates relational processing with Spark’s functional programming API. Built on our experience with Shark, Spark SQL lets Spark programmers leverage the benefits of relational processing (*e.g.*, declarative queries and optimized storage), and lets SQL users call complex analytics libraries in Spark (*e.g.*, machine learning). Compared to previous systems, Spark SQL makes two main additions. First, it offers much tighter integration between relational and procedural processing, through a declarative `DataFrame` API that integrates with procedural Spark code. Second, it includes a highly extensible optimizer, Catalyst, built using features of the Scala programming language, that makes it easy to add composable rules, control code generation, and define extension points. Using Catalyst, we have built a variety of features (*e.g.*, schema inference for JSON, machine learning types, and query federation to external databases) tailored for the complex needs of modern data analysis. We see Spark SQL as an evolution of both SQL-on-Spark and of Spark itself, offering richer APIs and optimizations while keeping the benefits of the Spark programming model.

Categories and Subject Descriptors

H.2 [Database Management]: Systems

Keywords

Databases; Data Warehouse; Machine Learning; Spark; Hadoop

1 Introduction

Big data applications require a mix of processing techniques, data sources and storage formats. The earliest systems designed for these workloads, such as MapReduce, gave users a powerful, but low-level, procedural programming interface. Programming such systems was onerous and required manual optimization by the user to achieve high performance. As a result, multiple new systems sought to provide a more productive user experience by offering relational interfaces to big data. Systems like Pig, Hive, Dremel and Shark [29, 36, 25, 38] all take advantage of declarative queries to provide richer automatic optimizations.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD’15, May 31–June 4, 2015, Melbourne, Victoria, Australia.
Copyright is held by the owner/author(s). Publication rights licensed to ACM.
ACM 978-1-4503-2758-9/15/05 ...\$15.00.
<http://dx.doi.org/10.1145/2723372.2742797>.

While the popularity of relational systems shows that users often prefer writing declarative queries, the relational approach is insufficient for many big data applications. First, users want to perform ETL to and from various data sources that might be semi- or unstructured, requiring custom code. Second, users want to perform advanced analytics, such as machine learning and graph processing, that are challenging to express in relational systems. In practice, we have observed that most data pipelines would ideally be expressed with a combination of both relational queries and complex procedural algorithms. Unfortunately, these two classes of systems—relational and procedural—have until now remained largely disjoint, forcing users to choose one paradigm or the other.

This paper describes our effort to combine both models in Spark SQL, a major new component in Apache Spark [39]. Spark SQL builds on our earlier SQL-on-Spark effort, called Shark. Rather than forcing users to pick between a relational or a procedural API, however, Spark SQL lets users seamlessly intermix the two.

Spark SQL bridges the gap between the two models through two contributions. First, Spark SQL provides a *DataFrame API* that can perform relational operations on both external data sources and Spark’s built-in distributed collections. This API is similar to the widely used data frame concept in R [32], but evaluates operations lazily so that it can perform relational optimizations. Second, to support the wide range of data sources and algorithms in big data, Spark SQL introduces a novel extensible optimizer called *Catalyst*. Catalyst makes it easy to add data sources, optimization rules, and data types for domains such as machine learning.

The `DataFrame` API offers rich relational/procedural integration within Spark programs. `DataFrames` are collections of structured records that can be manipulated using Spark’s procedural API, or using new relational APIs that allow richer optimizations. They can be created directly from Spark’s built-in distributed collections of Java/Python objects, enabling relational processing in existing Spark programs. Other Spark components, such as the machine learning library, take and produce `DataFrames` as well. `DataFrames` are more convenient and more efficient than Spark’s procedural API in many common situations. For example, they make it easy to compute multiple aggregates in one pass using a SQL statement, something that is difficult to express in traditional functional APIs. They also automatically store data in a columnar format that is significantly more compact than Java/Python objects. Finally, unlike existing data frame APIs in R and Python, `DataFrame` operations in Spark SQL go through a relational optimizer, Catalyst.

To support a wide variety of data sources and analytics workloads in Spark SQL, we designed an extensible query optimizer called Catalyst. Catalyst uses features of the Scala programming language, such as pattern-matching, to express composable rules in a Turing-complete language. It offers a general framework for transforming

Apache Spark

Apache DataFusion Comet

- Large scale distributed query processing
- Originated at UC Berkeley’s AMPLab



Spark SQL: Relational Data Processing in Spark

Michael Armbrust[†], Reynold S. Xin[†], Cheng Lian[†], Yin Huai[†], Davies Liu[†], Joseph K. Bradley[†], Xiangrui Meng[†], Tomer Kaftan[‡], Michael J. Franklin^{†‡}, Ali Ghodsi[†], Matei Zaharia^{†*}

[†]Databricks Inc. ^{*}MIT CSAIL [‡]AMPLab, UC Berkeley

ABSTRACT

Spark SQL is a new module in Apache Spark that integrates relational processing with Spark’s functional programming API. Built on our experience with Shark, Spark SQL lets Spark programmers leverage the benefits of relational processing (*e.g.*, declarative queries and optimized storage), and lets SQL users call complex analytics libraries in Spark (*e.g.*, machine learning). Compared to previous systems, Spark SQL makes two main additions. First, it offers much tighter integration between relational and procedural processing, through a declarative `DataFrame` API that integrates with procedural Spark code. Second, it includes a highly extensible optimizer, Catalyst, built using features of the Scala programming language, that makes it easy to add composable rules, control code generation, and define extension points. Using Catalyst, we have built a variety of features (*e.g.*, schema inference for JSON, machine learning types, and query federation to external databases) tailored for the complex needs of modern data analysis. We see Spark SQL as an evolution of both SQL-on-Spark and of Spark itself, offering richer APIs and optimizations while keeping the benefits of the Spark programming model.

Categories and Subject Descriptors

H.2 [Database Management]: Systems

Keywords

Databases; Data Warehouse; Machine Learning; Spark; Hadoop

1 Introduction

Big data applications require a mix of processing techniques, data sources and storage formats. The earliest systems designed for these workloads, such as MapReduce, gave users a powerful, but low-level, procedural programming interface. Programming such systems was onerous and required manual optimization by the user to achieve high performance. As a result, multiple new systems sought to provide a more productive user experience by offering relational interfaces to big data. Systems like Pig, Hive, Dremel and Shark [29, 36, 25, 38] all take advantage of declarative queries to provide richer automatic optimizations.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD’15, May 31–June 4, 2015, Melbourne, Victoria, Australia.
Copyright is held by the owner/author(s). Publication rights licensed to ACM.
ACM 978-1-4503-2758-9/15/05 ...\$15.00.
<http://dx.doi.org/10.1145/2723372.2742797>.

While the popularity of relational systems shows that users often prefer writing declarative queries, the relational approach is insufficient for many big data applications. First, users want to perform ETL to and from various data sources that might be semi- or unstructured, requiring custom code. Second, users want to perform advanced analytics, such as machine learning and graph processing, that are challenging to express in relational systems. In practice, we have observed that most data pipelines would ideally be expressed with a combination of both relational queries and complex procedural algorithms. Unfortunately, these two classes of systems—relational and procedural—have until now remained largely disjoint, forcing users to choose one paradigm or the other.

This paper describes our effort to combine both models in Spark SQL, a major new component in Apache Spark [39]. Spark SQL builds on our earlier SQL-on-Spark effort, called Shark. Rather than forcing users to pick between a relational or a procedural API, however, Spark SQL lets users seamlessly intermix the two.

Spark SQL bridges the gap between the two models through two contributions. First, Spark SQL provides a *DataFrame API* that can perform relational operations on both external data sources and Spark’s built-in distributed collections. This API is similar to the widely used data frame concept in R [32], but evaluates operations lazily so that it can perform relational optimizations. Second, to support the wide range of data sources and algorithms in big data, Spark SQL introduces a novel extensible optimizer called *Catalyst*. Catalyst makes it easy to add data sources, optimization rules, and data types for domains such as machine learning.

The `DataFrame` API offers rich relational/procedural integration within Spark programs. `DataFrames` are collections of structured records that can be manipulated using Spark’s procedural API, or using new relational APIs that allow richer optimizations. They can be created directly from Spark’s built-in distributed collections of Java/Python objects, enabling relational processing in existing Spark programs. Other Spark components, such as the machine learning library, take and produce `DataFrames` as well. `DataFrames` are more convenient and more efficient than Spark’s procedural API in many common situations. For example, they make it easy to compute multiple aggregates in one pass using a SQL statement, something that is difficult to express in traditional functional APIs. They also automatically store data in a columnar format that is significantly more compact than Java/Python objects. Finally, unlike existing data frame APIs in R and Python, `DataFrame` operations in Spark SQL go through a relational optimizer, Catalyst.

To support a wide variety of data sources and analytics workloads in Spark SQL, we designed an extensible query optimizer called Catalyst. Catalyst uses features of the Scala programming language, such as pattern-matching, to express composable rules in a Turing-complete language. It offers a general framework for transforming

Apache Spark

Apache DataFusion Comet

- Large scale distributed query processing
- Originated at UC Berkeley’s AMPLab
- Downsides:
 - Row-based query processing
 - JVM execution not as fast as native code
 - JVM garbage collection adds unpredictable latency



Spark SQL: Relational Data Processing in Spark

Michael Armbrust[†], Reynold S. Xin[†], Cheng Lian[†], Yin Huai[†], Davies Liu[†], Joseph K. Bradley[†], Xiangrui Meng[†], Tomer Kaftan[‡], Michael J. Franklin^{†‡}, Ali Ghodsi[†], Matei Zaharia^{†*}

[†]Databricks Inc. ^{*}MIT CSAIL [‡]AMPLab, UC Berkeley

ABSTRACT

Spark SQL is a new module in Apache Spark that integrates relational processing with Spark’s functional programming API. Built on our experience with Shark, Spark SQL lets Spark programmers leverage the benefits of relational processing (*e.g.*, declarative queries and optimized storage), and lets SQL users call complex analytics libraries in Spark (*e.g.*, machine learning). Compared to previous systems, Spark SQL makes two main additions. First, it offers much tighter integration between relational and procedural processing, through a declarative *DataFrame* API that integrates with procedural Spark code. Second, it includes a highly extensible optimizer, Catalyst, built using features of the Scala programming language, that makes it easy to add composable rules, control code generation, and define extension points. Using Catalyst, we have built a variety of features (*e.g.*, schema inference for JSON, machine learning types, and query federation to external databases) tailored for the complex needs of modern data analysis. We see Spark SQL as an evolution of both SQL-on-Spark and of Spark itself, offering richer APIs and optimizations while keeping the benefits of the Spark programming model.

Categories and Subject Descriptors

H.2 [Database Management]: Systems

Keywords

Databases; Data Warehouse; Machine Learning; Spark; Hadoop

1 Introduction

Big data applications require a mix of processing techniques, data sources and storage formats. The earliest systems designed for these workloads, such as MapReduce, gave users a powerful, but low-level, procedural programming interface. Programming such systems was onerous and required manual optimization by the user to achieve high performance. As a result, multiple new systems sought to provide a more productive user experience by offering relational interfaces to big data. Systems like Pig, Hive, Dremel and Shark [29, 36, 25, 38] all take advantage of declarative queries to provide richer automatic optimizations.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD’15, May 31–June 4, 2015, Melbourne, Victoria, Australia.
Copyright is held by the owner/author(s). Publication rights licensed to ACM.
ACM 978-1-4503-2758-9/15/05 ...\$15.00.
<http://dx.doi.org/10.1145/2723372.2742797>.

While the popularity of relational systems shows that users often prefer writing declarative queries, the relational approach is insufficient for many big data applications. First, users want to perform ETL to and from various data sources that might be semi- or unstructured, requiring custom code. Second, users want to perform advanced analytics, such as machine learning and graph processing, that are challenging to express in relational systems. In practice, we have observed that most data pipelines would ideally be expressed with a combination of both relational queries and complex procedural algorithms. Unfortunately, these two classes of systems—relational and procedural—have until now remained largely disjoint, forcing users to choose one paradigm or the other.

This paper describes our effort to combine both models in Spark SQL, a major new component in Apache Spark [39]. Spark SQL builds on our earlier SQL-on-Spark effort, called Shark. Rather than forcing users to pick between a relational or a procedural API, however, Spark SQL lets users seamlessly intermix the two.

Spark SQL bridges the gap between the two models through two contributions. First, Spark SQL provides a *DataFrame* API that can perform relational operations on both external data sources and Spark’s built-in distributed collections. This API is similar to the widely used data frame concept in R [32], but evaluates operations lazily so that it can perform relational optimizations. Second, to support the wide range of data sources and algorithms in big data, Spark SQL introduces a novel extensible optimizer called *Catalyst*. Catalyst makes it easy to add data sources, optimization rules, and data types for domains such as machine learning.

The *DataFrame* API offers rich relational/procedural integration within Spark programs. *DataFrames* are collections of structured records that can be manipulated using Spark’s procedural API, or using new relational APIs that allow richer optimizations. They can be created directly from Spark’s built-in distributed collections of Java/Python objects, enabling relational processing in existing Spark programs. Other Spark components, such as the machine learning library, take and produce *DataFrames* as well. *DataFrames* are more convenient and more efficient than Spark’s procedural API in many common situations. For example, they make it easy to compute multiple aggregates in one pass using a SQL statement, something that is difficult to express in traditional functional APIs. They also automatically store data in a columnar format that is significantly more compact than Java/Python objects. Finally, unlike existing data frame APIs in R and Python, *DataFrame* operations in Spark SQL go through a relational optimizer, Catalyst.

To support a wide variety of data sources and analytics workloads in Spark SQL, we designed an extensible query optimizer called Catalyst. Catalyst uses features of the Scala programming language, such as pattern-matching, to express composable rules in a Turing-complete language. It offers a general framework for transforming

Apache Spark

Apache DataFusion Comet

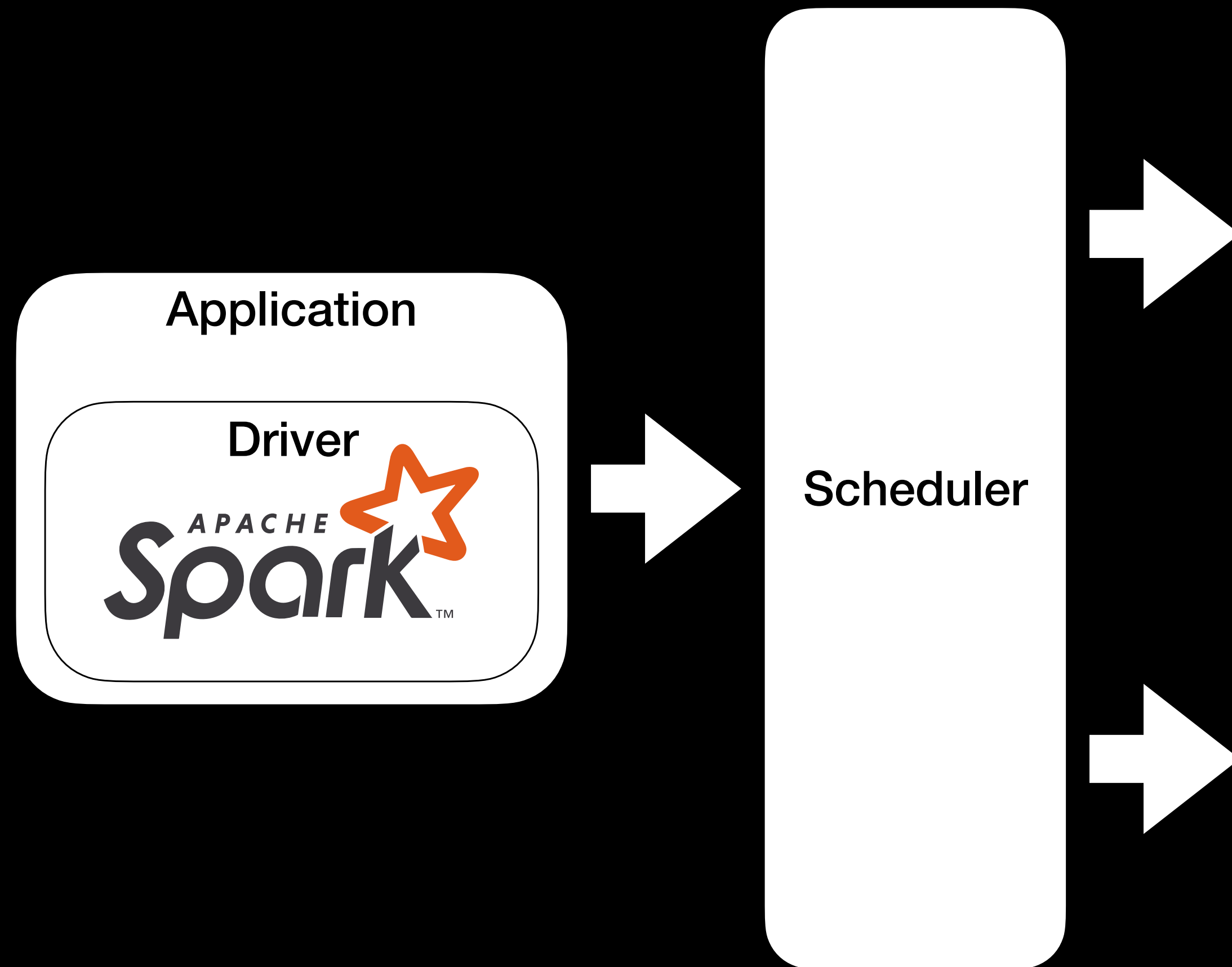
Apache Spark

Apache DataFusion Comet



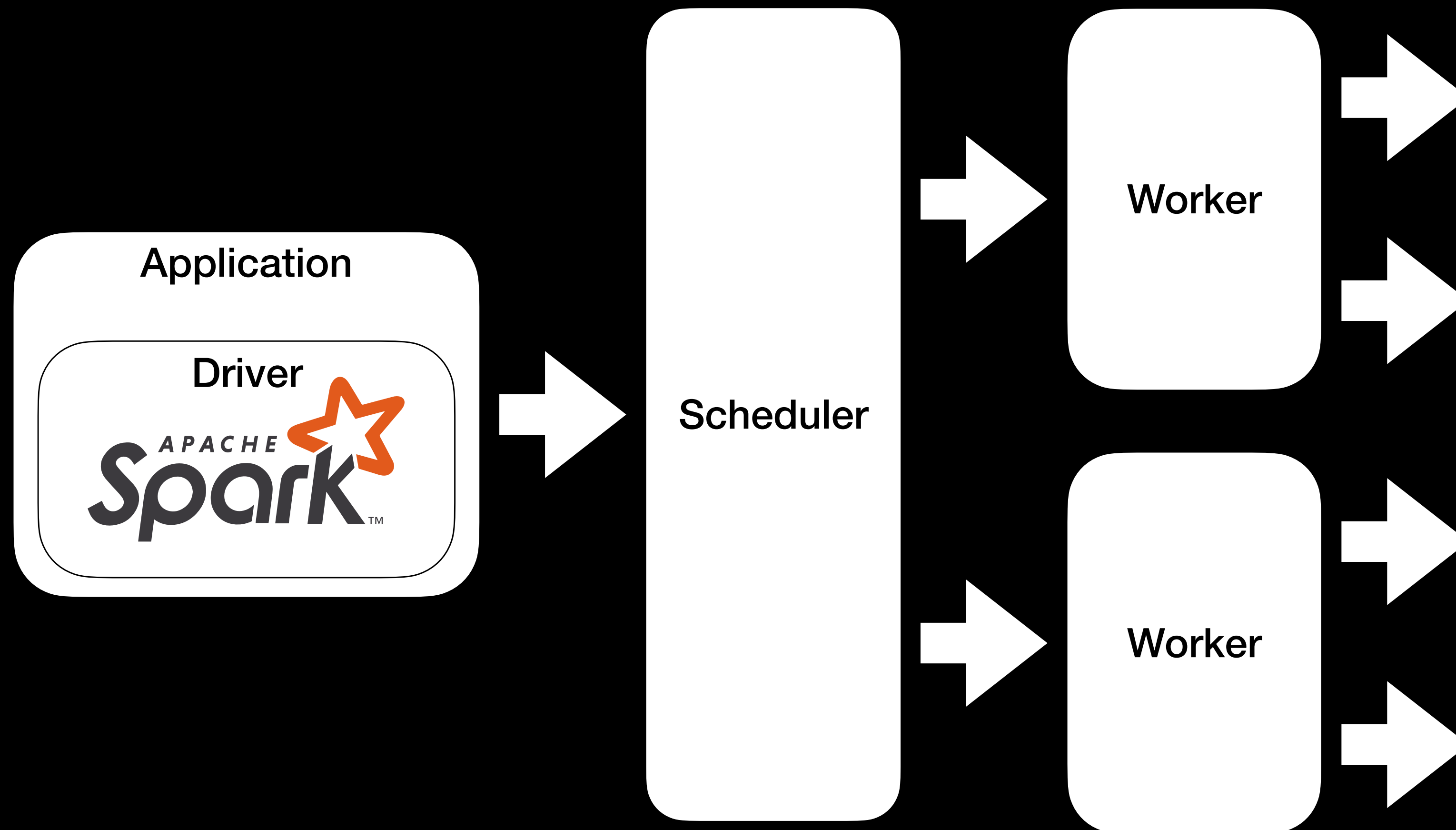
Apache Spark

Apache DataFusion Comet



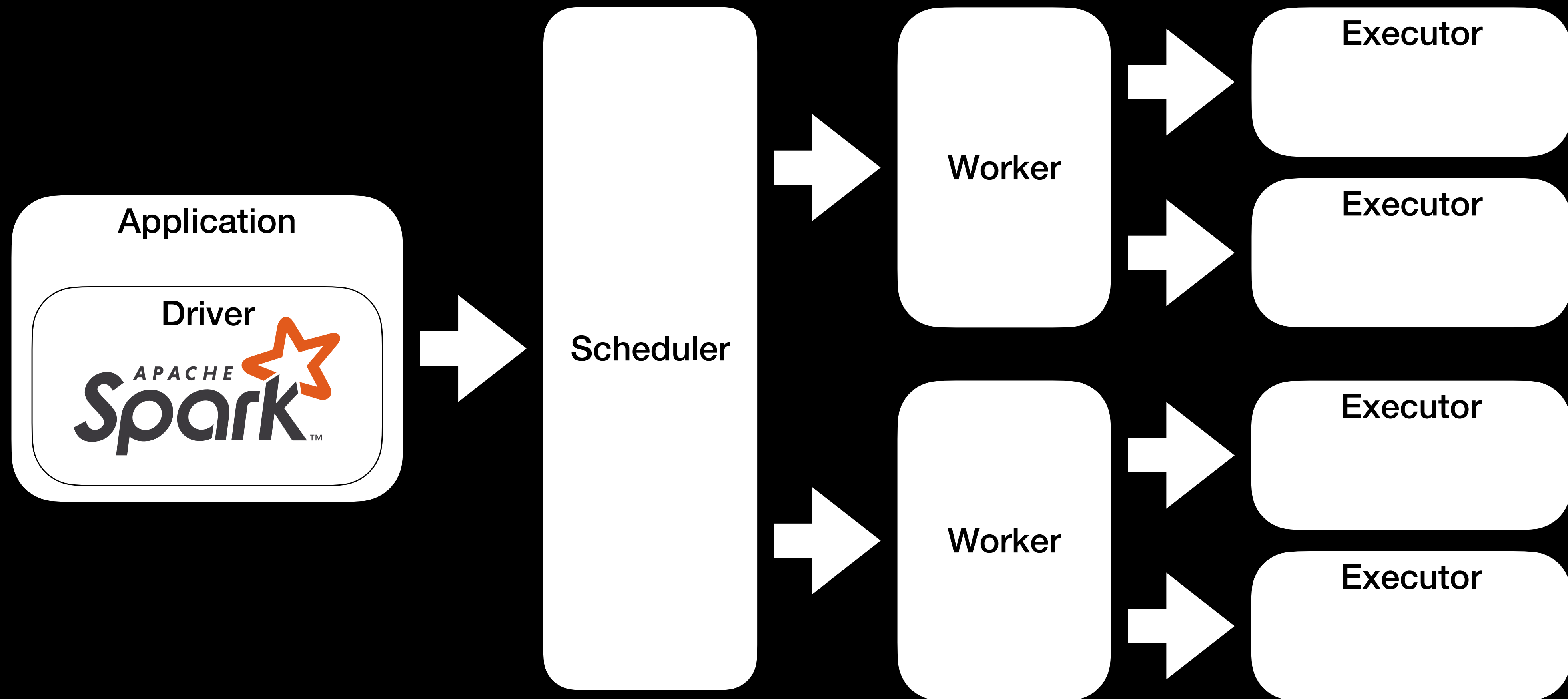
Apache Spark

Apache DataFusion Comet



Apache Spark

Apache DataFusion Comet



Overview

Apache DataFusion Comet

Overview

Apache DataFusion Comet

- Apache Spark accelerator based on Apache DataFusion

Overview

Apache DataFusion Comet

- Apache Spark accelerator based on Apache DataFusion
- Accelerate shuffle, data loading, operators, expressions, and functions

Overview

Apache DataFusion Comet

- Apache Spark accelerator based on Apache DataFusion
- Accelerate shuffle, data loading, operators, expressions, and functions
- Fall back to Spark for unsupported operations

Overview

Apache DataFusion Comet

- Apache Spark accelerator based on Apache DataFusion
- Accelerate shuffle, data loading, operators, expressions, and functions
- Fall back to Spark for unsupported operations
- Compatible with Spark 3.3, 3.4, 3.5. Experimental support for 4.0

Overview

Apache DataFusion Comet

- Apache Spark accelerator based on Apache DataFusion
- Accelerate shuffle, data loading, operators, expressions, and functions
- Fall back to Spark for unsupported operations
- Compatible with Spark 3.3, 3.4, 3.5. Experimental support for 4.0
- Donated to The Apache Software Foundation in February 2024

Apache Spark

Apache DataFusion Comet

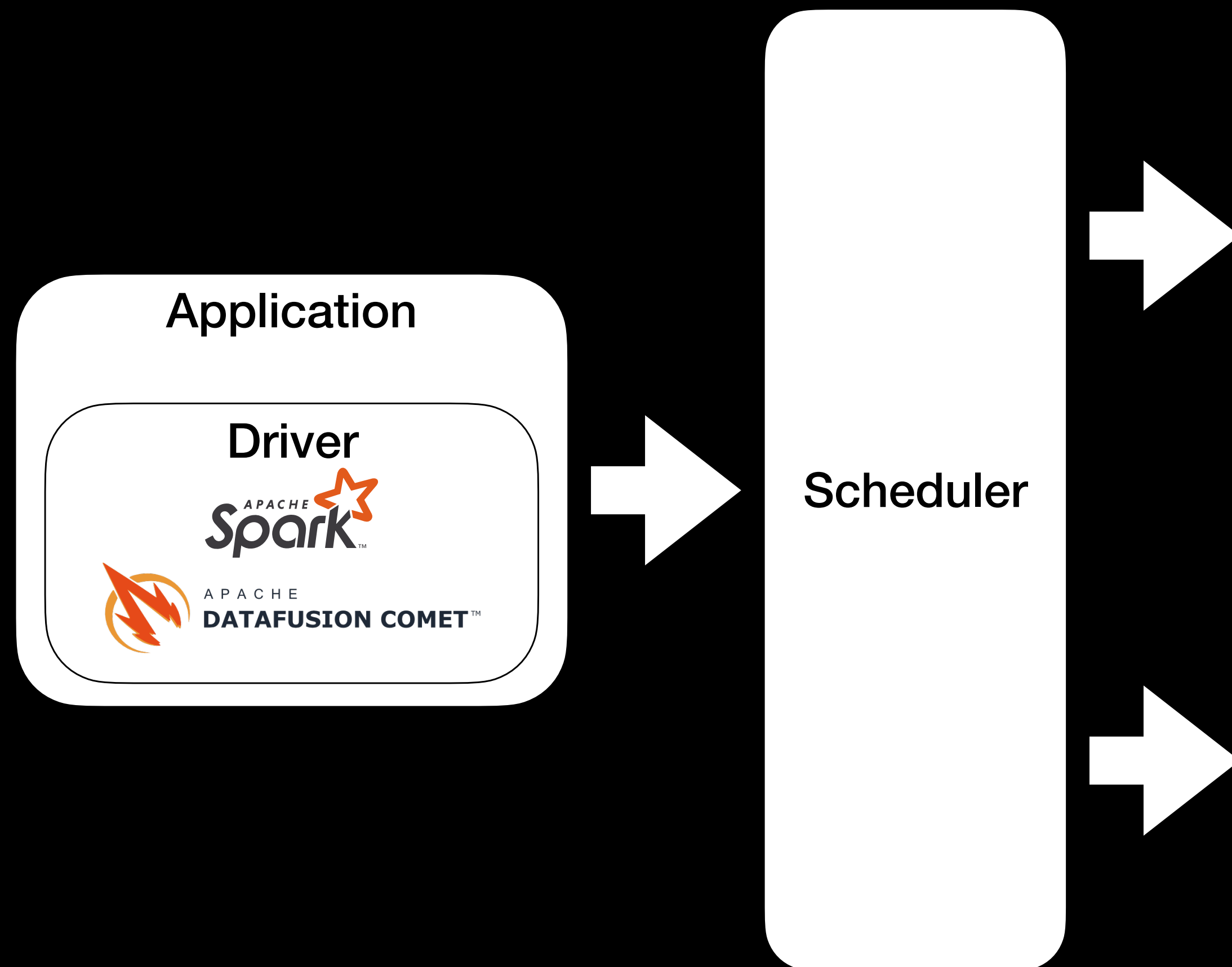
Apache Spark

Apache DataFusion Comet



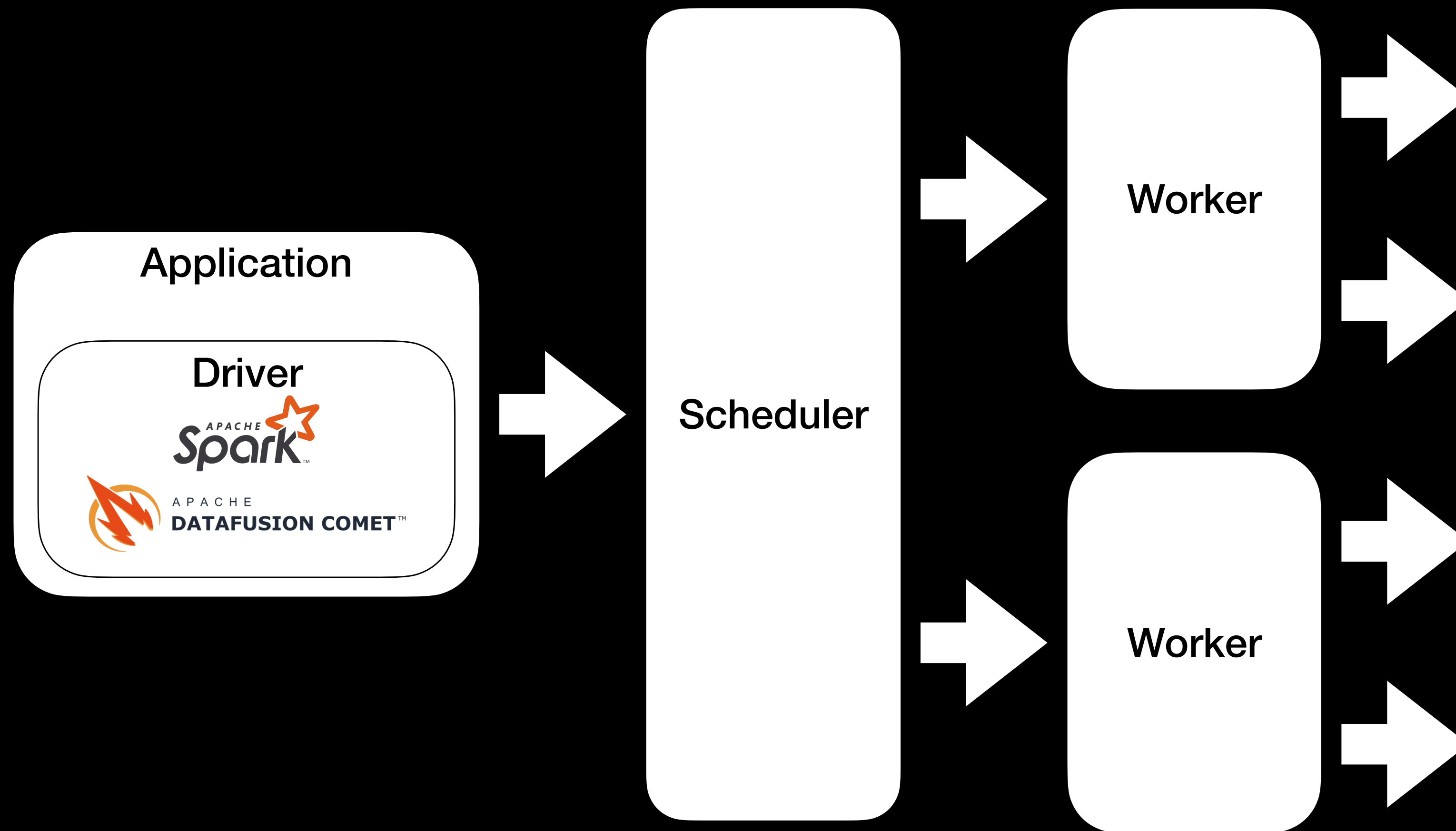
Apache Spark

Apache DataFusion Comet



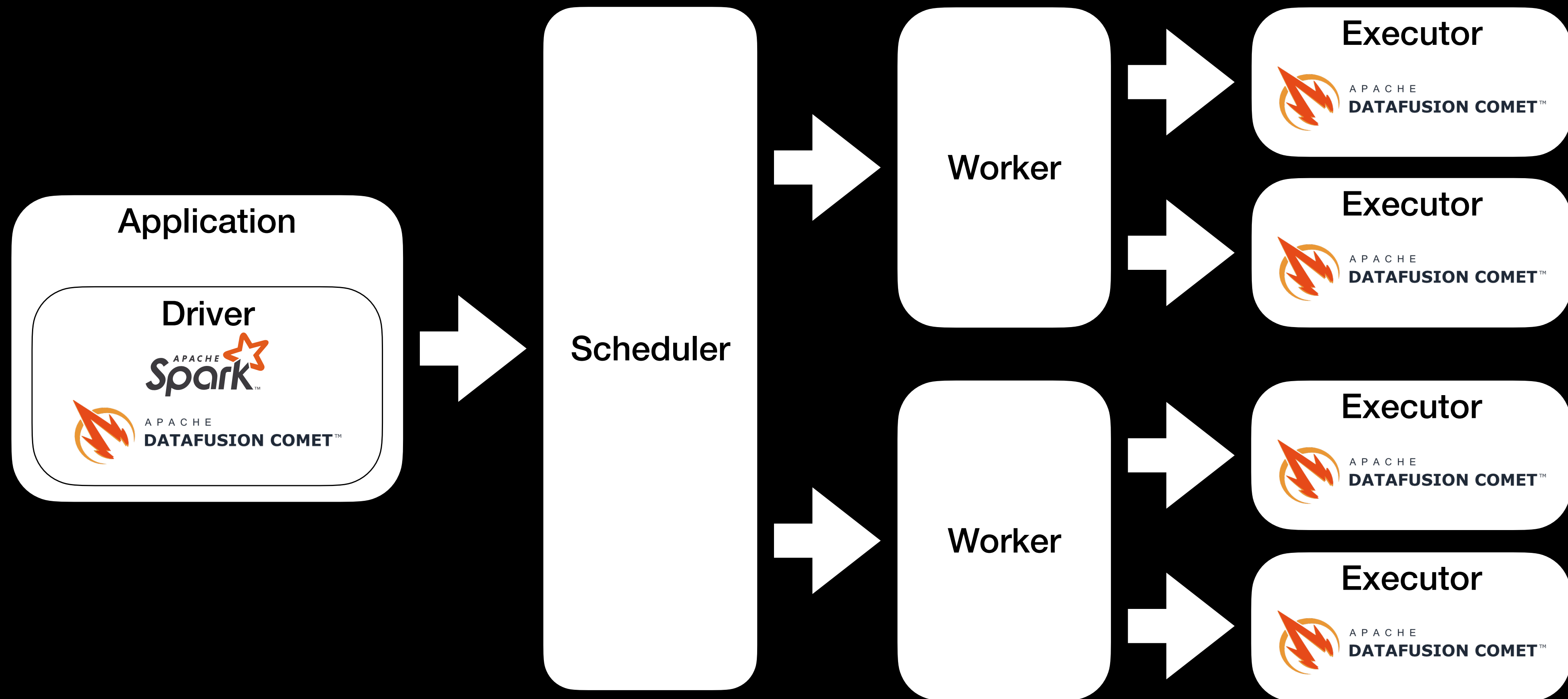
Apache Spark

Apache DataFusion Comet



Apache Spark

Apache DataFusion Comet



Databricks Photon

Apache DataFusion Comet

Photon: A Fast Query Engine for Lakehouse Systems

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepien, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia
photon-paper-authors@databricks.com
Databricks Inc.

ABSTRACT

Many organizations are shifting to a data management paradigm called the “Lakehouse,” which implements the functionality of structured data warehouses on top of unstructured data lakes. This presents new challenges for query execution engines. The execution engine needs to provide good performance on the raw uncurated datasets that are ubiquitous in data lakes, and excellent performance on structured data stored in popular columnar file formats like Apache Parquet. Toward these goals, we present Photon, a vectorized query engine for Lakehouse environments that we developed at Databricks. Photon can outperform existing cloud data warehouses in SQL workloads, but implements a more general execution framework that enables efficient processing of raw data and also enables Photon to support the Apache Spark API. We discuss the design choices we made in Photon (e.g., vectorization vs. code generation) and describe its integration with our existing SQL and Apache Spark runtimes, its task model, and its memory manager. Photon has accelerated some customer workloads by over 10× and has recently allowed Databricks to set a new audited performance record for the official 100TB TPC-DS benchmark.

ACM Reference Format:

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepien, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia . 2022. Photon: A Fast Query Engine for Lakehouse Systems. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD ’22)*, June 12–17, 2022, Philadelphia, PA, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3514221.3526054>

1 INTRODUCTION

Enterprises today store a vast majority of their data in scalable, elastic *data lakes* such as Amazon S3, Azure Data Lake Storage, and Google Cloud Storage. These data lakes hold raw, often uncurated datasets in open file formats such as Apache Parquet or Delta Lake [4, 18], and can be accessed with a variety of engines, such as Apache Spark and Presto [49, 58], to run workloads ranging

from SQL to machine learning. Traditionally, for the most demanding SQL workloads, enterprises have also moved a curated subset of their data into data warehouses to get high performance, governance and concurrency. However, this two-tier architecture is complex and expensive, as only a subset of data is available in the warehouse, and this data may be out of sync with the raw data due to issues in the extract, transform and load (ETL) process [19].

In response, many organizations are shifting to a data management architecture called the *Lakehouse* [19], which implements data warehouse features such as governance, ACID transactions and rich SQL support directly over a data lake. This single-tier approach promises to simplify data management, as users can govern and query *all* their data in a uniform way, and there are fewer ETL steps and query engines to manage. Recently, new storage layers such as Delta Lake [18] have enabled many of the management features of data warehouses—such as transactions and time travel—on data lakes, and have provided useful tools for optimizing storage access, such as data clustering and data skipping indices. However, maximizing the performance of Lakehouse workloads requires optimizing not only the storage layer, but also query processing.

This paper presents Photon, a new vectorized query engine we developed at Databricks for Lakehouse workloads that can execute queries written in either SQL or in Apache Spark’s DataFrame API [20]. Photon has already executed tens of millions of queries from hundreds of customers. With Photon, our customers have observed average speedups of 3× over our previous Databricks Runtime (an optimized engine based on Apache Spark), and maximum speedups of over 10×. Databricks also set an audited 100TB TPC-DS world record in November 2021 with Photon on a Lakehouse system using the Delta Lake format on Amazon S3, showing that state-of-the-art SQL performance is attainable with open data formats and commodity cloud storage.

Designing Photon required tackling two key challenges. First, unlike a traditional data warehouse engine, Photon needed to perform well on *raw, uncurated data*, which can include highly irregular datasets, poor physical layout, and large fields, all with no useful clustering or data statistics. Second, we wanted Photon to support, and be semantically compatible with, the existing Apache Spark DataFrame API that is widely used for data lake workloads. This was critical to deliver the Lakehouse promise of a *single* query engine with uniform semantics for all of an organization’s workloads, but created difficult engineering and testing challenges. Of course, subject to these two challenges, we wanted Photon to be as fast as possible. We describe how these two challenges led us to Photon’s design: a vectorized engine written in C++, that interfaces cleanly with Apache Spark’s memory manager and that includes a variety of optimizations for raw, uncurated data.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD ’22, June 12–17, 2022, Philadelphia, PA, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9249-5/22/06...\$15.00
<https://doi.org/10.1145/3514221.3526054>

Databricks Photon

Apache DataFusion Comet

- Apache Spark accelerator for Databricks

Photon: A Fast Query Engine for Lakehouse Systems

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepian, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia
photon-paper-authors@databricks.com
Databricks Inc.

ABSTRACT

Many organizations are shifting to a data management paradigm called the “Lakehouse,” which implements the functionality of structured data warehouses on top of unstructured data lakes. This presents new challenges for query execution engines. The execution engine needs to provide good performance on the raw uncurated datasets that are ubiquitous in data lakes, and excellent performance on structured data stored in popular columnar file formats like Apache Parquet. Toward these goals, we present Photon, a vectorized query engine for Lakehouse environments that we developed at Databricks. Photon can outperform existing cloud data warehouses in SQL workloads, but implements a more general execution framework that enables efficient processing of raw data and also enables Photon to support the Apache Spark API. We discuss the design choices we made in Photon (e.g., vectorization vs. code generation) and describe its integration with our existing SQL and Apache Spark runtimes, its task model, and its memory manager. Photon has accelerated some customer workloads by over 10× and has recently allowed Databricks to set a new audited performance record for the official 100TB TPC-DS benchmark.

ACM Reference Format:

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepian, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia . 2022. Photon: A Fast Query Engine for Lakehouse Systems. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD ’22)*, June 12–17, 2022, Philadelphia, PA, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3514221.3526054>

1 INTRODUCTION

Enterprises today store a vast majority of their data in scalable, elastic *data lakes* such as Amazon S3, Azure Data Lake Storage, and Google Cloud Storage. These data lakes hold raw, often uncurated datasets in open file formats such as Apache Parquet or Delta Lake [4, 18], and can be accessed with a variety of engines, such as Apache Spark and Presto [49, 58], to run workloads ranging

from SQL to machine learning. Traditionally, for the most demanding SQL workloads, enterprises have also moved a curated subset of their data into data warehouses to get high performance, governance and concurrency. However, this two-tier architecture is complex and expensive, as only a subset of data is available in the warehouse, and this data may be out of sync with the raw data due to issues in the extract, transform and load (ETL) process [19].

In response, many organizations are shifting to a data management architecture called the *Lakehouse* [19], which implements data warehouse features such as governance, ACID transactions and rich SQL support directly over a data lake. This single-tier approach promises to simplify data management, as users can govern and query *all* their data in a uniform way, and there are fewer ETL steps and query engines to manage. Recently, new storage layers such as Delta Lake [18] have enabled many of the management features of data warehouses—such as transactions and time travel—on data lakes, and have provided useful tools for optimizing storage access, such as data clustering and data skipping indices. However, maximizing the performance of Lakehouse workloads requires optimizing not only the storage layer, but also query processing.

This paper presents Photon, a new vectorized query engine we developed at Databricks for Lakehouse workloads that can execute queries written in either SQL or in Apache Spark’s DataFrame API [20]. Photon has already executed tens of millions of queries from hundreds of customers. With Photon, our customers have observed average speedups of 3× over our previous Databricks Runtime (an optimized engine based on Apache Spark), and maximum speedups of over 10×. Databricks also set an audited 100TB TPC-DS world record in November 2021 with Photon on a Lakehouse system using the Delta Lake format on Amazon S3, showing that state-of-the-art SQL performance is attainable with open data formats and commodity cloud storage.

Designing Photon required tackling two key challenges. First, unlike a traditional data warehouse engine, Photon needed to perform well on *raw, uncurated data*, which can include highly irregular datasets, poor physical layout, and large fields, all with no useful clustering or data statistics. Second, we wanted Photon to support, and be semantically compatible with, the existing Apache Spark DataFrame API that is widely used for data lake workloads. This was critical to deliver the Lakehouse promise of a *single* query engine with uniform semantics for all of an organization’s workloads, but created difficult engineering and testing challenges. Of course, subject to these two challenges, we wanted Photon to be as fast as possible. We describe how these two challenges led us to Photon’s design: a vectorized engine written in C++, that interfaces cleanly with Apache Spark’s memory manager and that includes a variety of optimizations for raw, uncurated data.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD ’22, June 12–17, 2022, Philadelphia, PA, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9249-5/22/06...\$15.00
<https://doi.org/10.1145/3514221.3526054>

Databricks Photon

Apache DataFusion Comet

- Apache Spark accelerator for Databricks
- Vectorized C++ engine

Photon: A Fast Query Engine for Lakehouse Systems

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepian, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia
photon-paper-authors@databricks.com
Databricks Inc.

ABSTRACT

Many organizations are shifting to a data management paradigm called the “Lakehouse,” which implements the functionality of structured data warehouses on top of unstructured data lakes. This presents new challenges for query execution engines. The execution engine needs to provide good performance on the raw uncurated datasets that are ubiquitous in data lakes, and excellent performance on structured data stored in popular columnar file formats like Apache Parquet. Toward these goals, we present Photon, a vectorized query engine for Lakehouse environments that we developed at Databricks. Photon can outperform existing cloud data warehouses in SQL workloads, but implements a more general execution framework that enables efficient processing of raw data and also enables Photon to support the Apache Spark API. We discuss the design choices we made in Photon (e.g., vectorization vs. code generation) and describe its integration with our existing SQL and Apache Spark runtimes, its task model, and its memory manager. Photon has accelerated some customer workloads by over 10× and has recently allowed Databricks to set a new audited performance record for the official 100TB TPC-DS benchmark.

ACM Reference Format:

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepian, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia . 2022. Photon: A Fast Query Engine for Lakehouse Systems. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD ’22)*, June 12–17, 2022, Philadelphia, PA, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3514221.3526054>

1 INTRODUCTION

Enterprises today store a vast majority of their data in scalable, elastic *data lakes* such as Amazon S3, Azure Data Lake Storage, and Google Cloud Storage. These data lakes hold raw, often uncurated datasets in open file formats such as Apache Parquet or Delta Lake [4, 18], and can be accessed with a variety of engines, such as Apache Spark and Presto [49, 58], to run workloads ranging

from SQL to machine learning. Traditionally, for the most demanding SQL workloads, enterprises have also moved a curated subset of their data into data warehouses to get high performance, governance and concurrency. However, this two-tier architecture is complex and expensive, as only a subset of data is available in the warehouse, and this data may be out of sync with the raw data due to issues in the extract, transform and load (ETL) process [19].

In response, many organizations are shifting to a data management architecture called the *Lakehouse* [19], which implements data warehouse features such as governance, ACID transactions and rich SQL support directly over a data lake. This single-tier approach promises to simplify data management, as users can govern and query *all* their data in a uniform way, and there are fewer ETL steps and query engines to manage. Recently, new storage layers such as Delta Lake [18] have enabled many of the management features of data warehouses—such as transactions and time travel—on data lakes, and have provided useful tools for optimizing storage access, such as data clustering and data skipping indices. However, maximizing the performance of Lakehouse workloads requires optimizing not only the storage layer, but also query processing.

This paper presents Photon, a new vectorized query engine we developed at Databricks for Lakehouse workloads that can execute queries written in either SQL or in Apache Spark’s DataFrame API [20]. Photon has already executed tens of millions of queries from hundreds of customers. With Photon, our customers have observed average speedups of 3× over our previous Databricks Runtime (an optimized engine based on Apache Spark), and maximum speedups of over 10×. Databricks also set an audited 100TB TPC-DS world record in November 2021 with Photon on a Lakehouse system using the Delta Lake format on Amazon S3, showing that state-of-the-art SQL performance is attainable with open data formats and commodity cloud storage.

Designing Photon required tackling two key challenges. First, unlike a traditional data warehouse engine, Photon needed to perform well on *raw, uncurated data*, which can include highly irregular datasets, poor physical layout, and large fields, all with no useful clustering or data statistics. Second, we wanted Photon to support, and be semantically compatible with, the existing Apache Spark DataFrame API that is widely used for data lake workloads. This was critical to deliver the Lakehouse promise of a *single* query engine with uniform semantics for all of an organization’s workloads, but created difficult engineering and testing challenges. Of course, subject to these two challenges, we wanted Photon to be as fast as possible. We describe how these two challenges led us to Photon’s design: a vectorized engine written in C++, that interfaces cleanly with Apache Spark’s memory manager and that includes a variety of optimizations for raw, uncurated data.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD ’22, June 12–17, 2022, Philadelphia, PA, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9249-5/22/06...\$15.00
<https://doi.org/10.1145/3514221.3526054>

Databricks Photon

Apache DataFusion Comet

- Apache Spark accelerator for Databricks
- Vectorized C++ engine
- Discusses design trade-offs:
 - JIT compilation vs. interpretation
 - Validity bitmaps vs. position vectors

Photon: A Fast Query Engine for Lakehouse Systems

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepian, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia
photon-paper-authors@databricks.com
Databricks Inc.

ABSTRACT

Many organizations are shifting to a data management paradigm called the “Lakehouse,” which implements the functionality of structured data warehouses on top of unstructured data lakes. This presents new challenges for query execution engines. The execution engine needs to provide good performance on the raw uncurated datasets that are ubiquitous in data lakes, and excellent performance on structured data stored in popular columnar file formats like Apache Parquet. Toward these goals, we present Photon, a vectorized query engine for Lakehouse environments that we developed at Databricks. Photon can outperform existing cloud data warehouses in SQL workloads, but implements a more general execution framework that enables efficient processing of raw data and also enables Photon to support the Apache Spark API. We discuss the design choices we made in Photon (e.g., vectorization vs. code generation) and describe its integration with our existing SQL and Apache Spark runtimes, its task model, and its memory manager. Photon has accelerated some customer workloads by over 10× and has recently allowed Databricks to set a new audited performance record for the official 100TB TPC-DS benchmark.

ACM Reference Format:

Alexander Behm, Shoumik Palkar, Utkarsh Agarwal, Timothy Armstrong, David Cashman, Ankur Dave, Todd Greenstein, Shant Hovsepian, Ryan Johnson, Arvind Sai Krishnan, Paul Leventis, Ala Luszczak, Prashanth Menon, Mostafa Mokhtar, Gene Pang, Sameer Paranjpye, Greg Rahn, Bart Samwel, Tom van Bussel, Herman van Hovell, Maryann Xue, Reynold Xin, Matei Zaharia . 2022. Photon: A Fast Query Engine for Lakehouse Systems. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*, June 12–17, 2022, Philadelphia, PA, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3514221.3526054>

1 INTRODUCTION

Enterprises today store a vast majority of their data in scalable, elastic *data lakes* such as Amazon S3, Azure Data Lake Storage, and Google Cloud Storage. These data lakes hold raw, often uncurated datasets in open file formats such as Apache Parquet or Delta Lake [4, 18], and can be accessed with a variety of engines, such as Apache Spark and Presto [49, 58], to run workloads ranging

from SQL to machine learning. Traditionally, for the most demanding SQL workloads, enterprises have also moved a curated subset of their data into data warehouses to get high performance, governance and concurrency. However, this two-tier architecture is complex and expensive, as only a subset of data is available in the warehouse, and this data may be out of sync with the raw data due to issues in the extract, transform and load (ETL) process [19].

In response, many organizations are shifting to a data management architecture called the *Lakehouse* [19], which implements data warehouse features such as governance, ACID transactions and rich SQL support directly over a data lake. This single-tier approach promises to simplify data management, as users can govern and query *all* their data in a uniform way, and there are fewer ETL steps and query engines to manage. Recently, new storage layers such as Delta Lake [18] have enabled many of the management features of data warehouses—such as transactions and time travel—on data lakes, and have provided useful tools for optimizing storage access, such as data clustering and data skipping indices. However, maximizing the performance of Lakehouse workloads requires optimizing not only the storage layer, but also query processing.

This paper presents Photon, a new vectorized query engine we developed at Databricks for Lakehouse workloads that can execute queries written in either SQL or in Apache Spark’s DataFrame API [20]. Photon has already executed tens of millions of queries from hundreds of customers. With Photon, our customers have observed average speedups of 3× over our previous Databricks Runtime (an optimized engine based on Apache Spark), and maximum speedups of over 10×. Databricks also set an audited 100TB TPC-DS world record in November 2021 with Photon on a Lakehouse system using the Delta Lake format on Amazon S3, showing that state-of-the-art SQL performance is attainable with open data formats and commodity cloud storage.

Designing Photon required tackling two key challenges. First, unlike a traditional data warehouse engine, Photon needed to perform well on *raw, uncurated data*, which can include highly irregular datasets, poor physical layout, and large fields, all with no useful clustering or data statistics. Second, we wanted Photon to support, and be semantically compatible with, the existing Apache Spark DataFrame API that is widely used for data lake workloads. This was critical to deliver the Lakehouse promise of a *single* query engine with uniform semantics for all of an organization’s workloads, but created difficult engineering and testing challenges. Of course, subject to these two challenges, we wanted Photon to be as fast as possible. We describe how these two challenges led us to Photon’s design: a vectorized engine written in C++, that interfaces cleanly with Apache Spark’s memory manager and that includes a variety of optimizations for raw, uncurated data.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD '22, June 12–17, 2022, Philadelphia, PA, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9249-5/22/06...\$15.00
<https://doi.org/10.1145/3514221.3526054>

Challenges

Apache DataFusion Comet

Challenges

Apache DataFusion Comet

- Spark is a moving target:

Challenges

Apache DataFusion Comet

- Spark is a moving target:
 - Behavior differences between Spark releases

Challenges

Apache DataFusion Comet

- Spark is a moving target:
 - Behavior differences between Spark releases
 - Configurations change casting behavior

Challenges

Apache DataFusion Comet

- Spark is a moving target:
 - Behavior differences between Spark releases
 - Configurations change casting behavior
 - ANSI SQL enforced in Spark 4.0

Challenges

Apache DataFusion Comet

- Spark is a moving target:
 - Behavior differences between Spark releases
 - Configurations change casting behavior
 - ANSI SQL enforced in Spark 4.0
- Data representation differences:

Challenges

Apache DataFusion Comet

- Spark is a moving target:
 - Behavior differences between Spark releases
 - Configurations change casting behavior
 - ANSI SQL enforced in Spark 4.0
- Data representation differences:
 - Spark uses Java's `BigDecimal`, Arrow uses `Decimal128`

Challenges

Apache DataFusion Comet

- Spark is a moving target:
 - Behavior differences between Spark releases
 - Configurations change casting behavior
 - ANSI SQL enforced in Spark 4.0
- Data representation differences:
 - Spark uses Java's `BigDecimal`, Arrow uses `Decimal128`
 - Spark converts unsigned types to the next size signed type

TPC-H, single-node, partitioned Parquet

Apache DataFusion Comet

TPC-H, single-node, partitioned Parquet

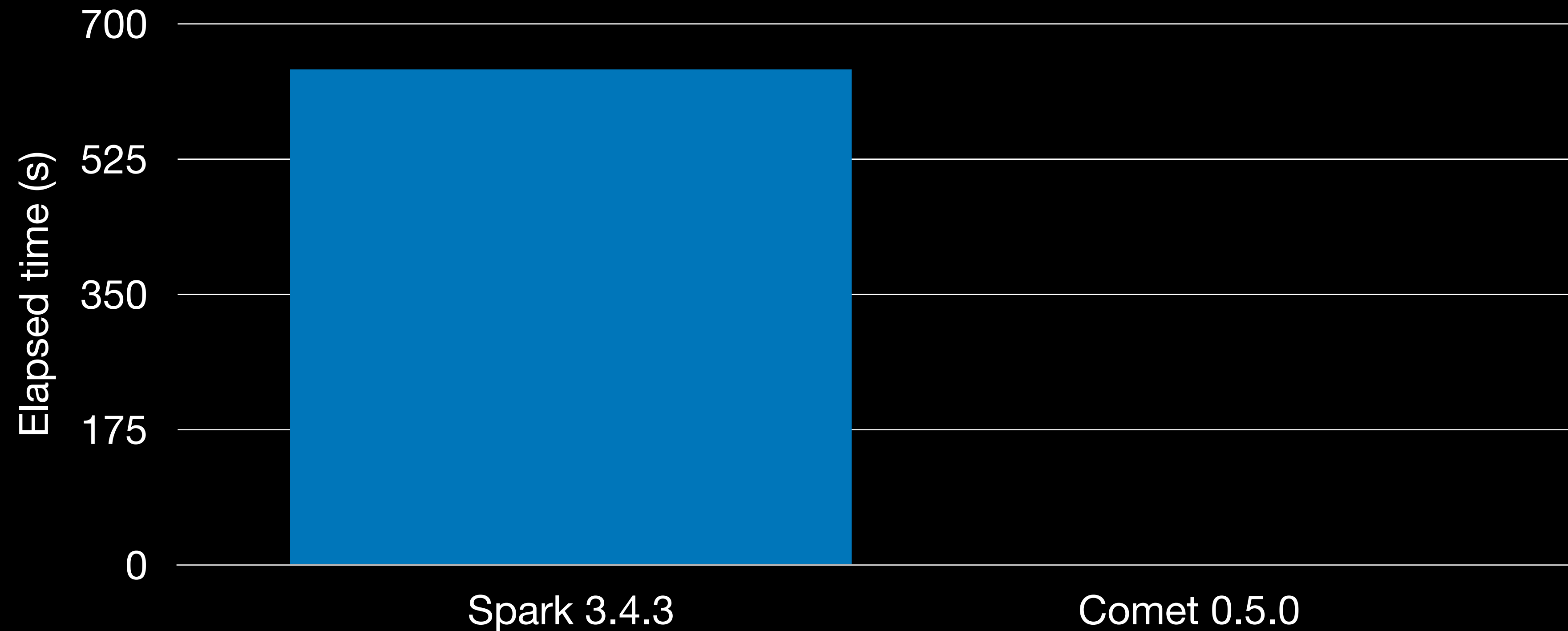
Apache DataFusion Comet



Source: [Apache DataFusion Comet 0.5.0 Release](#)

TPC-H, single-node, partitioned Parquet

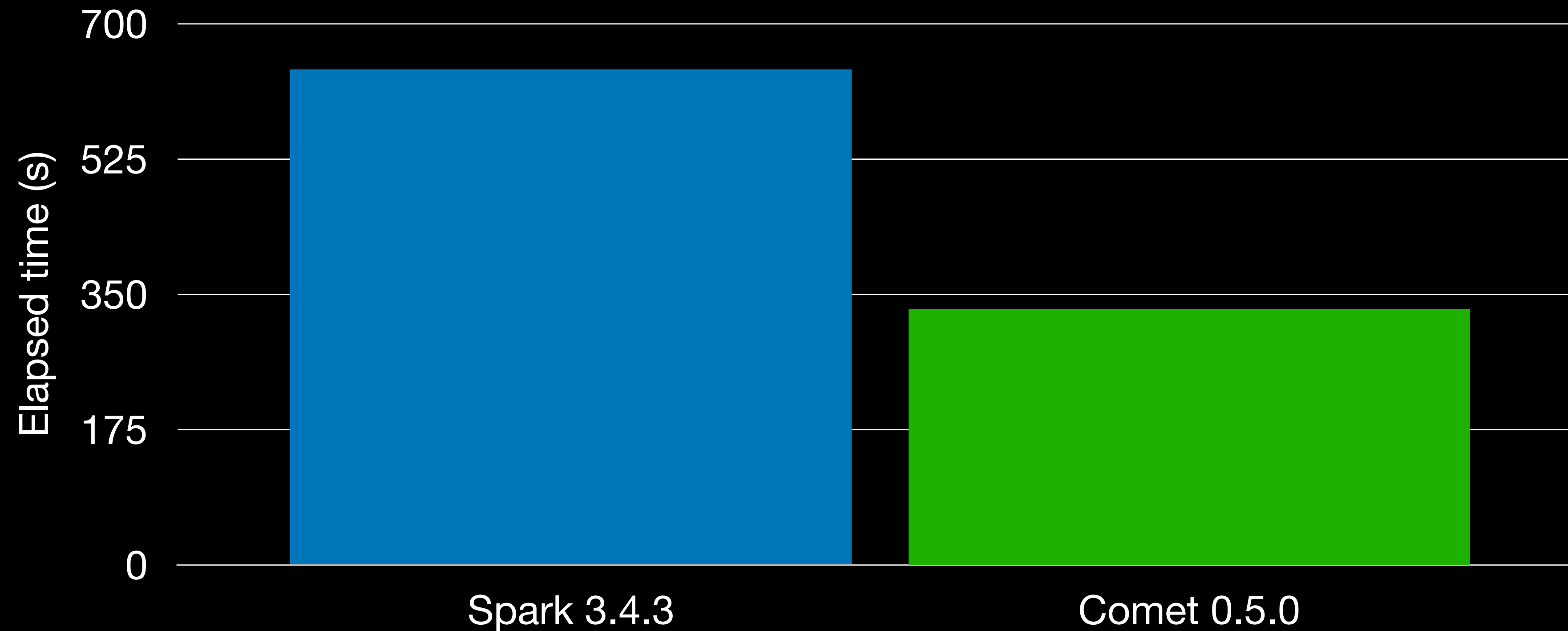
Apache DataFusion Comet



Source: [Apache DataFusion Comet 0.5.0 Release](#)

TPC-H, single-node, partitioned Parquet

Apache DataFusion Comet



Source: [Apache DataFusion Comet 0.5.0 Release](#)

Roadmap

Apache DataFusion Comet

Roadmap

Apache DataFusion Comet

- Add fuzz testing to CI

Roadmap

Apache DataFusion Comet

- Add fuzz testing to CI
- Complex Type Support
 - e.g., arrays, maps, structs
- Use DataFusion's Parquet scan instead of Comet Scan
- Support more native functions and expressions

Roadmap

Apache DataFusion Comet

- Add fuzz testing to CI
- Complex Type Support
 - e.g., arrays, maps, structs
- Use DataFusion's Parquet scan instead of Comet Scan
- Support more native functions and expressions
- Explore `StringView` performance benefits

How to Get Involved

Online Open-Source Communities

How to Get Involved

Online Open-Source Communities

How to Get Involved

- GitHub:
 - <https://github.com/apache/datafusion>
 - <https://github.com/apache/datafusion-comet>
 - <https://github.com/apache/arrow-rs>

Online Open-Source Communities

How to Get Involved

- GitHub:
 - <https://github.com/apache/datafusion>
 - <https://github.com/apache/datafusion-comet>
 - <https://github.com/apache/arrow-rs>
- [Apache DataFusion Community Meeting](#)
 - Wednesdays @ 11am ET

Online Open-Source Communities

How to Get Involved

- GitHub:
 - <https://github.com/apache/datafusion>
 - <https://github.com/apache/datafusion-comet>
 - <https://github.com/apache/arrow-rs>
- [Apache DataFusion Community Meeting](#)
 - Wednesdays @ 11am ET
- ASF Slack: <https://infra.apache.org/slack.html>

Online DBMS Resources

How to Get Involved



Online DBMS Resources

How to Get Involved

- Andy Pavlo @ CMU:



Online DBMS Resources

How to Get Involved

- Andy Pavlo @ CMU:
 - [Intro to DBMSs \(15-445\)](#)



Online DBMS Resources

How to Get Involved

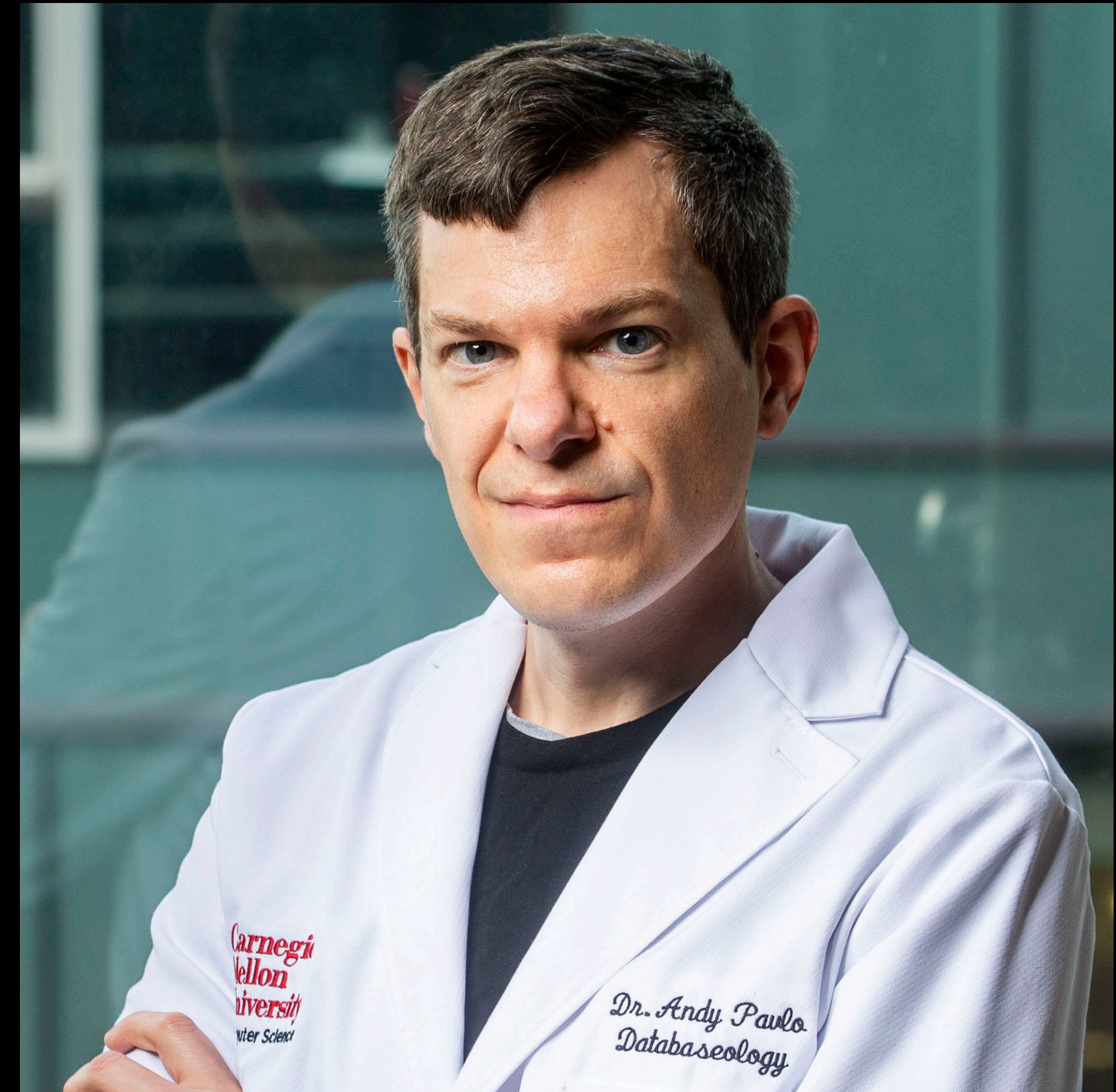
- Andy Pavlo @ CMU:
 - [Intro to DBMSs \(15-445\)](#)
 - [Advanced DBMSs \(15-721\)](#)



Online DBMS Resources

How to Get Involved

- Andy Pavlo @ CMU:
 - [Intro to DBMSs \(15-445\)](#)
 - [Advanced DBMSs \(15-721\)](#)
 - [Special Topics \(15-799\)](#):



Online DBMS Resources

How to Get Involved

- Andy Pavlo @ CMU:
 - [Intro to DBMSs \(15-445\)](#)
 - [Advanced DBMSs \(15-721\)](#)
 - [Special Topics \(15-799\)](#):
 - Optimizers, self-driving DBMSs



Online DBMS Resources

How to Get Involved

- Andy Pavlo @ CMU:
 - [Intro to DBMSs \(15-445\)](#)
 - [Advanced DBMSs \(15-721\)](#)
 - [Special Topics \(15-799\)](#):
 - Optimizers, self-driving DBMSs
 - [CMU-DB YouTube](#)



Online DBMS Resources

How to Get Involved

- Andy Pavlo @ CMU:
 - [Intro to DBMSs \(15-445\)](#)
 - [Advanced DBMSs \(15-721\)](#)
 - [Special Topics \(15-799\)](#):
 - Optimizers, self-driving DBMSs
 - [CMU-DB YouTube](#)
 - Lectures, seminars



Online Systems Communities

How to Get Involved

Online Systems Communities

How to Get Involved

- Discord:
 - Software Internals: <https://discord.gg/243d8pfGB9>
 - DataFusion / Arrow Rust: <https://discord.gg/geUyQt6DXJ>
 - DC Systems: <https://discord.gg/W6gvyDy8wP>

Online Systems Communities

How to Get Involved

- Discord:
 - Software Internals: <https://discord.gg/243d8pfGB9>
 - DataFusion / Arrow Rust: <https://discord.gg/geUyQt6DXJ>
 - DC Systems: <https://discord.gg/W6gvvDy8wP>
- Please share more!

Takeaways

Putting Theory Into Practice

Thank you to Andy Grove and Andrew Lamb for their support in creating this talk.

Takeaways

Putting Theory Into Practice

- Database management systems (DBMSs), their academic communities, and their open-source communities are awesome!

Thank you to Andy Grove and Andrew Lamb for their support in creating this talk.

Takeaways

Putting Theory Into Practice

- Database management systems (DBMSs), their academic communities, and their open-source communities are awesome!
- Apache DataFusion is a high-performance, embeddable analytics engine

Thank you to Andy Grove and Andrew Lamb for their support in creating this talk.

Takeaways

Putting Theory Into Practice

- Database management systems (DBMSs), their academic communities, and their open-source communities are awesome!
- Apache DataFusion is a high-performance, embeddable analytics engine
- Apache DataFusion Comet uses DataFusion to accelerate Apache Spark

Thank you to Andy Grove and Andrew Lamb for their support in creating this talk.

Takeaways

Putting Theory Into Practice

- Database management systems (DBMSs), their academic communities, and their open-source communities are awesome!
- Apache DataFusion is a high-performance, embeddable analytics engine
- Apache DataFusion Comet uses DataFusion to accelerate Apache Spark
- Reach out if you want to chat: <https://mattbutrovi.ch>

Thank you to Andy Grove and Andrew Lamb for their support in creating this talk.